app/src/main/kotlin/com/lirui/oneline/MainActivity.kt

```
package com.lirui.oneline

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.activity.result.contract.ActivityResultContracts
import androidx.activity.viewModels
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.core.content.ContextCompat
import com.lirui.oneline.focus.FocusEngine
import com.lirui.oneline.focus.FocusNotifier
import com.lirui.oneline.keepalive.DeviceCompat
import com.lirui.oneline.reminder.NotificationHelper
import com.lirui.oneline.reminder.ReminderScheduler
import com.lirui.oneline.reminder.ReminderWatchdogWorker
import com.lirui.oneline.ui.MainViewModel
import com.lirui.oneline.ui.screen.PrivacyConsentDialog
import com.lirui.oneline.ui.screen.PrivacyPolicyScreen
import com.lirui.oneline.ui.screen.TodoApp
import com.lirui.oneline.ui.theme.OnelineTheme
import com.lirui.oneline.util.SettingsRepo

class MainActivity : androidx.activity.ComponentActivity() {

    private val vm: MainViewModel by viewModels()

    /**
     * 从通知点进来的待办 id（-1 = 无）。放在 Activity 字段而不是 Composable state：
     * [onNewIntent]（singleTask 复用实例时走这里）也要能写它，而那边没有重组上下文。
     * 用 compose runtime 的 mutableStateOf 只是为了让 setContent 里的读取是响应式的。
     */
    private val pendingTodoId = mutableStateOf(-1L)

    /** 从专注通知点进来（true = 打开专注页）。与 [pendingTodoId] 同一套消费/复位方式。 */
    private val pendingFocus = mutableStateOf(false)
```

```
private val requestPerms = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { }

/**
 * 解析「点通知打开 App」的 intent：action = OPEN_TODO 且带 todoId 时记录，
 * 交给 TodoApp 滚到对应行并短暂高亮（v0.9.3 补丁，P2-4）。
 *
 * 在此之前这个 extra 是**死数据** —— 通知一直带着 todoId，但没有任何一侧读过。
 */
private fun handleIntent(intent: Intent?) {
    // 专注通知（进行中 / 已完成）点进来 → 直接铺开专注页。
    // 这两条通知的 action 是 FocusNotifier.ACTION_OPEN_FOCUS，不带 todoId。
    if (intent?.action == FocusNotifier.ACTION_OPEN_FOCUS) {
        pendingFocus.value = true
        return
    }
    if (intent?.action != NotificationHelper.ACTION_OPEN_TODO) return
    val id = intent.getLongExtra(NotificationHelper.EXTRA_TODO_ID, -1L)
    // 测试通知的 todoId 是 0（不对应任何待办），忽略。
    if (id > 0L) pendingTodoId.value = id
}

/** singleTask：已有实例复用走这里（按 Home 键后再点通知）。 */
override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)
    // 必须替换 instance intent：否则下次 getIntent() 拿到的还是旧 action，
    // 旋屏重建后会重新定位到上一条通知的待办。
    setIntent(intent)
    handleIntent(intent)
}

/**
 * 申请通知权限（Android 13+）。
 *
 * ★ v1.0.0 起**只能在用户同意隐私政策之后调用**。原来是 onCreate 里直接申请 ——
 *   「未征得同意先申请权限」是商店审核的常见拒审理由（个人信息保护法要求
 *   处理个人信息前先告知并取得同意，权限申请属于处理行为的前置动作）。
 */
private fun requestNotificationPermissionIfNeeded() {
    if (Build.VERSION.SDK_INT < 33) return
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
        == PackageManager.PERMISSION_GRANTED
    ) return
    runCatching { requestPerms.launch(arrayOf(Manifest.permission.POST_NOTIFICATIONS)) }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
```

```
        enableEdgeToEdge()

        // intent 解析与权限无关，放在同意门控之外：从通知点进来时要能定位到那一条。
        handleIntent(intent)

        setContent {
            val appContext = applicationContext
            var accepted by remember
{ mutableStateOf(SettingsRepo.privacyAccepted(appContext)) }
            var showPolicy by remember { mutableStateOf(false) }

            // ── 未同意隐私政策：只渲染弹窗，主界面完全不组合 ──
            // 这样能保证「同意之前不申请任何权限、不排任何闹钟、不产生任何系统副作用」。
            if (!accepted) {
                if (showPolicy) {
                    // 看政策时把同意弹窗整个撤掉，两个 Dialog 不叠着显示：
                    // 叠着会让返回键归属含糊，而且关掉政策页后弹窗重新出现，语义更清楚。
                    PrivacyPolicyScreen(onBack = { showPolicy = false })
                } else {
                    Box(
                        modifier = Modifier
                            .fillMaxSize()
                            .background(MaterialTheme.colorScheme.surface)
                    ) {}
                    PrivacyConsentDialog(
                        onAgree = {
                            SettingsRepo.setPrivacyAccepted(appContext)
                            accepted = true
                        },
                        // 不同意就真的退出 —— 不能只关弹窗然后继续用（审核明确要求）。
                        onDisagree = { finish() },
                        onViewPolicy = { showPolicy = true }
                    )
                }
                return@setContent
            }

            // ── 已同意：做一次「碰系统」的初始化，然后进主界面 ──
            // 幂等：冷启动 / 进程重建都会跑，重复排闹钟只是白排一次（见 ReminderScheduler 说明）。
            LaunchedEffect(accepted) {
                requestNotificationPermissionIfNeeded()
                // 冷启动对账（L2 自愈的第三道）：用户真正打开 App 时，把 DB 里"应该有"的闹钟重排一遍。
                // 覆盖的场景：被一键清理 force-stop 过、刚升级过、从备份恢复过 —— 这些情况下
                // AlarmManager 里的条目已经没了，只有重新 set 才能恢复。
                ReminderScheduler.scheduleAllActive(appContext)
                ReminderWatchdogWorker.ensureScheduled(appContext)
                // 专注同理：闹钟条目不落盘，冷启动 / 升级 / 重启后要把「还在跑的专注」
                // 的到点闹钟和常驻通知重新挂上，否则计时会一直走、永远不响。
                FocusEngine.restore(appContext, System.currentTimeMillis())
            }
```

```
            // Theme preference state (follow-system / light / dark).
            var themeMode by remember { mutableStateOf(SettingsRepo.getThemeMode(this)) }
            val onSetThemeMode: (Int) -> Unit = { mode ->
                themeMode = mode
                SettingsRepo.setThemeMode(this, mode)
            }
            // Colour palette (one of the built-in M3E / vivid presets).
            var paletteId by remember { mutableStateOf(SettingsRepo.paletteId(this)) }
            val onSetPalette: (Int) -> Unit = { id ->
                paletteId = id
                SettingsRepo.setPaletteId(this, id)
            }
            // Dark-mode card tint style (标准暗色 / 淡彩 / 不染色).
            var darkCard by remember { mutableStateOf(SettingsRepo.darkCardStyleId(this)) }
            val onSetDarkCard: (Int) -> Unit = { id ->
                darkCard = id
                SettingsRepo.setDarkCardStyleId(this, id)
            }
            // Reminder switches.
            var remindersEnabled by remember
{ mutableStateOf(SettingsRepo.remindersEnabled(this)) }
            var remindAtDue by remember { mutableStateOf(SettingsRepo.remindAtDue(this)) }
            var remindInAdvance by remember
{ mutableStateOf(SettingsRepo.remindInAdvance(this)) }
            // Overdue/divergence switches.
            var overdueNag by remember { mutableStateOf(SettingsRepo.overdueNagEnabled(this)) }
            var overdueInterval by remember
{ mutableStateOf(SettingsRepo.overdueIntervalHours(this)) }
            // Home grouping: board order / visibility + the buckets inside 有日期.
            var boardOrder by remember { mutableStateOf(SettingsRepo.homeBoardOrder(this)) }
            var hiddenBoards by remember { mutableStateOf(SettingsRepo.hiddenBoards(this)) }
            var homeBuckets by remember { mutableStateOf(SettingsRepo.homeBuckets(this)) }
            var defaultMinute by remember
{ mutableStateOf(SettingsRepo.defaultMinuteOfDay(this)) }
            var autoRepeat by remember { mutableStateOf(SettingsRepo.autoRepeatEnabled(this)) }
            val onRescheduleReminders: () -> Unit = {
                ReminderScheduler.scheduleAllActive(applicationContext)
            }
            OnelineTheme(
                darkTheme = when (themeMode) {
                    SettingsRepo.DARK -> true
                    SettingsRepo.LIGHT -> false
                    else -> androidx.compose.foundation.isSystemInDarkTheme()
                },
                paletteId = paletteId,
                darkCardStyle = com.lirui.oneline.ui.theme.DarkCardStyle.byId(darkCard)
            ) {
                val context = LocalContext.current
                TodoApp(
                    vm,
                    onExactAlarmSettings = {
                        // 带 package Uri 才能直达本应用；不认这个 action 的 ROM 会自动退到应用详情
页。
                        DeviceCompat.openExactAlarmSettings(context)
                    },
```

```
themeMode = themeMode,
onSetThemeMode = onSetThemeMode,
paletteId = paletteId,
onSetPalette = onSetPalette,
darkCardStyleId = darkCard,
onSetDarkCardStyle = onSetDarkCard,
remindersEnabled = remindersEnabled,
onSetRemindersEnabled = { on ->
    remindersEnabled = on
    SettingsRepo.setRemindersEnabled(this, on)
    onRescheduleReminders()
},
remindAtDue = remindAtDue,
onSetRemindAtDue = { on ->
    remindAtDue = on
    SettingsRepo.setRemindAtDue(this, on)
    onRescheduleReminders()
},
remindInAdvance = remindInAdvance,
onSetRemindInAdvance = { on ->
    remindInAdvance = on
    SettingsRepo.setRemindInAdvance(this, on)
    onRescheduleReminders()
},
overdueNag = overdueNag,
onSetOverdueNag = { on ->
    overdueNag = on
    SettingsRepo.setOverdueNagEnabled(this, on)
    onRescheduleReminders()
},
overdueInterval = overdueInterval,
onSetOverdueInterval = { hours ->
    overdueInterval = hours
    SettingsRepo.setOverdueIntervalHours(this, hours)
    onRescheduleReminders()
},
boardOrder = boardOrder,
onSetBoardOrder = { order ->
    boardOrder = order
    SettingsRepo.setHomeBoardOrder(this, order)
},
hiddenBoards = hiddenBoards,
onSetHiddenBoards = { hidden ->
    hiddenBoards = hidden
    SettingsRepo.setHiddenBoards(this, hidden)
},
homeBuckets = homeBuckets,
onSetHomeBuckets = { buckets ->
    homeBuckets = buckets
    SettingsRepo.setHomeBuckets(this, buckets)
```

```
                    },
                    defaultMinuteOfDay = defaultMinute,
                    onSetDefaultMinuteOfDay = { mo ->
                        defaultMinute = mo
                        SettingsRepo.setDefaultMinuteOfDay(this, mo)
                        // Existing active todos keep their current due; new date-only ones use
it.
                        onRescheduleReminders()
                    },
                    autoRepeat = autoRepeat,
                    onSetAutoRepeat = { on ->
                        autoRepeat = on
                        SettingsRepo.setAutoRepeatEnabled(this, on)
                    },
                    onRescheduleReminders = onRescheduleReminders,
                    // 点通知打开：滚到对应待办并高亮（P2-4）。
                    pendingTodoId = pendingTodoId.value,
                    onPendingTodoHandled = { pendingTodoId.value = -1L },
                    // 点专注通知打开：直接进专注页。
                    pendingFocus = pendingFocus.value,
                    onPendingFocusHandled = { pendingFocus.value = false }
                )
            }
        }
    }
}
```

app/src/main/kotlin/com/lirui/oneline/OnelineApp.kt

```
package com.lirui.oneline

import android.app.Application
import com.lirui.oneline.focus.FocusNotifier
import com.lirui.oneline.reminder.NotificationHelper
import com.lirui.oneline.reminder.ReminderWatchdogWorker

class OnelineApp : Application() {
    override fun onCreate() {
        super.onCreate()
        // Create notification channels up front (idempotent).
        NotificationHelper.init(this)
        // 专注的两个渠道：倒计时（静音常驻）/ 完成（要响）。
        FocusNotifier.init(this)
        // 兜底巡检（每 12h）。KEEP 语义，重复调用无副作用。
        // 放在 Application 而不是 MainActivity：它要覆盖"开机后一直没打开过 App"的区间，
        // 而那种情况下 MainActivity 根本不会执行。
        ReminderWatchdogWorker.ensureScheduled(this)
    }
}
```

app/src/main/kotlin/com/lirui/oneline/data/AppDatabase.kt

```
package com.lirui.oneline.data

import android.content.Context
```

```
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import androidx.room.migration.Migration
import androidx.sqlite.db.SupportSQLiteDatabase
import com.lirui.oneline.data.model.FocusSession
import com.lirui.oneline.data.model.Todo

@Database(
    entities = [Todo::class, FocusSession::class],
    version = 7,
    exportSchema = true
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun todoDao(): TodoDao
    abstract fun focusDao(): FocusDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        /** Add [repeatDays] (default 0 = no-repeat) to the todos table. */
        private val MIGRATION_1_2 = object : Migration(1, 2) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE todos ADD COLUMN repeatDays INTEGER NOT NULL DEFAULT 0")
            }
        }

        /** Add [detail] (free text after the event title) to the todos table. */
        private val MIGRATION_2_3 = object : Migration(2, 3) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE todos ADD COLUMN detail TEXT NOT NULL DEFAULT ''")
            }
        }

        /**
         * Add [startAtMillis]（时间段开始日；可空，null = 单日待办）。
         * 老数据全部是单日，startAtMillis 自然为 null，行为零变化。
         */
        private val MIGRATION_3_4 = object : Migration(3, 4) {
            override fun migrate(db: SupportSQLiteDatabase) {
                db.execSQL("ALTER TABLE todos ADD COLUMN startAtMillis INTEGER")
            }
        }

        /**
         * Add [Todo.deletedAt]（历史列，现在没有任何代码写它；见 [Todo] 的类注释）。
         *
         * ★★ 这一句必须带 `DEFAULT NULL`。踩过的坑（0.9.3 一打开就闪退）：
         *    实体上写的是 `@ColumnInfo(defaultValue = "NULL")`，Room 迁移完会拿**实际列定义**
```

```
 *   跟实体期望的比对 —— 而 `ALTER TABLE ... ADD COLUMN x INTEGER`（不写 DEFAULT）
 *   在 `PRAGMA table_info` 里读到的 dflt_value 是**空**，只有显式写
 *   `DEFAULT NULL` 才是字符串 `'NULL'`。两者不等 → Room 抛
 *   `Migration didn't properly handle: todos(...)` → 启动即崩，且编译期完全看不出来。
 *
 * ★ 幂等：已经装过出问题那一版、迁移中途失败但**没有回滚**的设备，列可能已经存在。
 *   直接 ALTER 会抛 "duplicate column name"，所以先查 PRAGMA 再决定加不加。
 */
private val MIGRATION_4_5 = object : Migration(4, 5) {
    override fun migrate(db: SupportSQLiteDatabase) {
        if (!hasColumn(db, "todos", "deletedAt")) {
            db.execSQL("ALTER TABLE todos ADD COLUMN deletedAt INTEGER DEFAULT NULL")
        }
    }
}

/**
 * Add [Todo.endAtMillis]（时间区间结束点；可空，null = 普通待办）。
 * 老数据全 null，行为零变化。同样必须带 `DEFAULT NULL`（坑见 [MIGRATION_4_5]）。
 */
private val MIGRATION_5_6 = object : Migration(5, 6) {
    override fun migrate(db: SupportSQLiteDatabase) {
        if (!hasColumn(db, "todos", "endAtMillis")) {
            db.execSQL("ALTER TABLE todos ADD COLUMN endAtMillis INTEGER DEFAULT NULL")
        }
    }
}

/**
 * 新增 [FocusSession] 表（v1.0.1 专注计时）。
 *
 * 纯建表，**不动 todos** —— 老用户的待办一条都不会少，也不存在列默认值那种坑
 * （见 [MIGRATION_4_5]）。`IF NOT EXISTS` 是幂等保险：迁移中途失败但没回滚的设备
 * 再跑一次不会抛 "table already exists"。
 *
 * ★ 列定义必须和实体逐字对齐（类型、NOT NULL、主键自增），否则 Room 迁移后校验
 *   不通过会直接抛 `Migration didn't properly handle: focus_sessions(...)` ——
 *   和 0.9.3 那次闪退是同一类错误，只是这次错在建表而不是加列。
 */
private val MIGRATION_6_7 = object : Migration(6, 7) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS `focus_sessions` (" +
                "`id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, " +
                "`label` TEXT NOT NULL, " +
                "`startedAtMillis` INTEGER NOT NULL, " +
                "`endedAtMillis` INTEGER NOT NULL, " +
                "`plannedSeconds` INTEGER NOT NULL, " +
                "`actualSeconds` INTEGER NOT NULL, " +
                "`completed` INTEGER NOT NULL, " +
```

```
                    "`createdAt` INTEGER NOT NULL)"
                )
            }
        }

        private fun hasColumn(db: SupportSQLiteDatabase, table: String, column: String):
Boolean {
            db.query("PRAGMA table_info($table)").use { cursor ->
                val nameIndex = cursor.getColumnIndex("name")
                if (nameIndex < 0) return false
                while (cursor.moveToNext()) {
                    if (cursor.getString(nameIndex) == column) return true
                }
            }
            return false
        }

        fun get(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                INSTANCE ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "oneline.db"
                )
                    .addMigrations(
                        MIGRATION_1_2, MIGRATION_2_3, MIGRATION_3_4,
                        MIGRATION_4_5, MIGRATION_5_6, MIGRATION_6_7
                    )
                    .build().also { INSTANCE = it }
            }
        }
    }
}
```

app/src/main/kotlin/com/lirui/oneline/data/FocusDao.kt

```
package com.lirui.oneline.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import com.lirui.oneline.data.model.FocusSession
import kotlinx.coroutines.flow.Flow

@Dao
interface FocusDao {

    /** 全部记录，新的在前。统计与列表共用这一份数据（见 FocusStats）。 */
    @Query("SELECT * FROM focus_sessions ORDER BY startedAtMillis DESC")
    fun observeAll(): Flow<List<FocusSession>>

    @Query("SELECT * FROM focus_sessions ORDER BY startedAtMillis DESC")
    suspend fun getAll(): List<FocusSession>
```

```
    @Insert
    suspend fun insert(session: FocusSession): Long

    @Query("DELETE FROM focus_sessions WHERE id = :id")
    suspend fun deleteById(id: Long)
}
```

app/src/main/kotlin/com/lirui/oneline/data/FocusRepository.kt

```
package com.lirui.oneline.data

import android.content.Context
import com.lirui.oneline.data.model.FocusSession
import kotlinx.coroutines.flow.Flow

/**
 * 专注记录的唯一读写入口。
 *
 * 和 [TodoRepository] 一样薄：这里不做业务判断（阈值、是否记入统计都在
 * `focus.FocusRules` / `FocusViewModel` 里），只负责"写进去"和"读出来"。
 */
class FocusRepository(context: Context) {

    private val dao = AppDatabase.get(context).focusDao()

    val all: Flow<List<FocusSession>> = dao.observeAll()

    suspend fun allOnce(): List<FocusSession> = dao.getAll()

    /**
     * 记一次专注。
     *
     * @param actualSeconds 已经过 `FocusRules` 判定为"该记"的秒数。
     *                      调用方保证它 > 0 —— 这里不再兜底，0 秒的记录没有意义，
     *                      真写进去会让统计里出现"专注 0 秒"的脏行。
     */
    suspend fun record(
        label: String,
        startedAtMillis: Long,
        endedAtMillis: Long,
        plannedSeconds: Int,
        actualSeconds: Int,
        completed: Boolean
    ): Long = dao.insert(
        FocusSession(
            label = label.trim(),
            startedAtMillis = startedAtMillis,
            endedAtMillis = endedAtMillis,
            plannedSeconds = plannedSeconds,
            actualSeconds = actualSeconds,
            completed = completed,
```

```
            createdAt = System.currentTimeMillis()
        )
    )

    suspend fun delete(id: Long) = dao.deleteById(id)
}
```

app/src/main/kotlin/com/lirui/oneline/data/TodoDao.kt

```
package com.lirui.oneline.data

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.Query
import androidx.room.Update
import com.lirui.oneline.data.model.Todo
import kotlinx.coroutines.flow.Flow

@Dao
interface TodoDao {

    @Query("SELECT * FROM todos WHERE archived = 0 ORDER BY COALESCE(dueAtMillis,
253402300799999) ASC, createdAt ASC")
    fun observeActive(): Flow<List<Todo>>

    @Query("SELECT * FROM todos WHERE archived = 1 ORDER BY completedAt DESC")
    fun observeArchived(): Flow<List<Todo>>

    @Query("SELECT * FROM todos WHERE id = :id")
    suspend fun getById(id: Long): Todo?

    @Insert
    suspend fun insert(todo: Todo): Long

    @Update
    suspend fun update(todo: Todo)

    @Delete
    suspend fun delete(todo: Todo)

    @Query("DELETE FROM todos WHERE id = :id")
    suspend fun deleteById(id: Long)

    @Query("SELECT * FROM todos WHERE archived = 0 ORDER BY COALESCE(dueAtMillis,
253402300799999) ASC, createdAt ASC")
    suspend fun getActiveAll(): List<Todo>

    @Query("UPDATE todos SET archived = 1 WHERE id = :id")
    suspend fun markArchivedById(id: Long)

    @Query("UPDATE todos SET archived = 0 WHERE id = :id")
    suspend fun unarchiveById(id: Long)
```

```
    @Query("UPDATE todos SET title = :title, detail = :detail, dueAtMillis = :dueAtMillis,
startAtMillis = :startAtMillis, endAtMillis = :endAtMillis, hasExactTime = :hasExactTime,
repeatDays = :repeatDays WHERE id = :id")
    suspend fun updateContent(id: Long, title: String, detail: String, dueAtMillis: Long?,
startAtMillis: Long?, endAtMillis: Long?, hasExactTime: Boolean, repeatDays: Int)
}
```

app/src/main/kotlin/com/lirui/oneline/data/TodoRepository.kt

```
package com.lirui.oneline.data

import android.content.Context
import com.lirui.oneline.data.model.Todo
import com.lirui.oneline.parser.ParsedTodo
import com.lirui.oneline.reminder.NotificationDismissScheduler
import com.lirui.oneline.reminder.NotificationHelper
import com.lirui.oneline.reminder.ReminderEngine
import com.lirui.oneline.reminder.ReminderScheduler
import com.lirui.oneline.util.AppTime
import com.lirui.oneline.util.SettingsRepo
import kotlinx.coroutines.flow.Flow
import java.time.LocalDate
import java.time.LocalTime
import java.time.ZonedDateTime

/**
 * Single facade over the DB + reminder scheduler + parse conversion.
 */
class TodoRepository(private val context: Context) {

    private val dao = AppDatabase.get(context).todoDao()

    val active: Flow<List<Todo>> = dao.observeActive()
    val archived: Flow<List<Todo>> = dao.observeArchived()

    suspend fun addFromParsed(parsed: ParsedTodo): Long {
        val due = parsedToDue(parsed)
        val start = parsedToStart(parsed)
        val end = parsedToEnd(parsed)
        val id = dao.insert(Todo(
            title = parsed.cleanTitle,
            detail = parsed.detail.orEmpty(),
            dueAtMillis = due?.first,
            startAtMillis = start,
            endAtMillis = end,
            hasExactTime = parsed.hasExactTime,
            repeatDays = parsed.repeatDays
        ))
        due?.let { (epoch, exact) ->
            ReminderScheduler.scheduleBlocking(context, id, epoch, exact, start, end)
        }
        return id
    }

    /**
```

```
 * Build the concrete due epoch from a [ParsedTodo].
 * Returns null for a no-date item. Defaults missing time-of-day to the user-set default
 * (SettingsRepo.defaultMinuteOfDay, default 09:00).
 */
private fun parsedToDue(parsed: ParsedTodo): Pair<Long, Boolean>? {
    val zone = AppTime.ZONE
    val date = parsed.dueDate
    if (date == null) return null
    val time = parsed.dueTime ?: defaultTime(parsed.dueDate)
    val zdt = ZonedDateTime.of(date, time, zone)
    return zdt.toInstant().toEpochMilli() to parsed.hasExactTime
}

/**
 * 时间段的开始日 epoch；单日待办（[ParsedTodo.startDate] == null）返回 null。
 * 开始日只按「日」存（当天 00:00）—— 时刻只作用于结束日（dueAtMillis）。
 */
private fun parsedToStart(parsed: ParsedTodo): Long? {
    val date = parsed.startDate ?: return null
    // 开始日 == 结束日（"周一到周一"/"24 日到 24 日"）就是单日待办，不要落成退化区间，
    // 否则列表会一直挂着「最后一天」。
    if (parsed.dueDate == null || parsed.dueDate == date) return null
    return date.atStartOfDay(AppTime.ZONE).toInstant().toEpochMilli()
}

/**
 * 时间区间的结束点 epoch（#1 方案 B）；[ParsedTodo.endTime] == null 返回 null。
 *
 * 与 [dueAtMillis]（开始点，提醒在这响）**同一天**；跨午夜的写法（「晚上 11 点到 1 点」）
 * 结束点落在次日 —— 用户写的就是这个意思，显示与逾期判定都按结束点走。
 */
private fun parsedToEnd(parsed: ParsedTodo): Long? {
    val end = parsed.endTime ?: return null
    val date = parsed.dueDate ?: return null
    val start = parsed.dueTime
    val d = if (start != null && end.isBefore(start)) date.plusDays(1) else date
    return ZonedDateTime.of(d, end, AppTime.ZONE).toInstant().toEpochMilli()
}

/** Default time-of-day for a date-only todo — the user-configurable default. */
private fun defaultTime(date: LocalDate?): LocalTime {
    // coerceIn 与 ReminderEngine.startReminder 的防护风格对齐：脏设置值不会炸 LocalTime.of。
    // 当前写入路径只有设置页 4 个固定档位（实际不可达），属防御（P3-1）。
    val mo = SettingsRepo.defaultMinuteOfDay(context).coerceIn(0, 1439)
    return LocalTime.of(mo / 60, mo % 60)
}

/**
 * Settings test button: a 「通知测试」 todo due at the next whole minute.
 *
```

```
 * 标题只写「通知测试」，**不要**把「该办了」写进来 —— 那是提醒状态语，
 * 由 `NotificationHelper.composeTitle` 拼成 `通知测试 🚀🚀该办了⏳⏳`
 * （状态语与 emoji 现在都可在设置页改，默认值即此）。
 * 写进标题会变成「通知测试 该办了 🚀🚀该办了⏳⏳」。
 */
suspend fun addTestInOneMinute(): Long {
    val zone = AppTime.ZONE
    val due = ZonedDateTime.now(zone).withSecond(0).withNano(0).plusMinutes(1)
    val epoch = due.toInstant().toEpochMilli()
    val id = dao.insert(Todo(
        title = "通知测试",
        detail = "下一分钟到期",
        dueAtMillis = epoch,
        hasExactTime = true
    ))
    ReminderScheduler.scheduleBlocking(context, id, epoch, true)
    return id
}

/**
 * 归档一条待办（重复任务改为顺延）。
 *
 * @return false = 这条待办已经不在了（被删除 / 被别处处理过）。
 *   调用方必须据此收起"归档中"状态，否则 UI 会永远停在勾选态。
 */
suspend fun archive(id: Long): Boolean {
    val todo = dao.getById(id) ?: return false
    // A repeating todo rolls to its next occurrence instead of being archived.
    if (todo.repeatDays > 0 && SettingsRepo.autoRepeatEnabled(context)) {
        rollForward(todo)
    } else {
        dao.update(todo.copy(archived = true, completedAt = System.currentTimeMillis()))
        ReminderScheduler.cancelForId(context, id)
    }
    // ★ 归档后排一个「10 秒后撤通知」，**三个入口共用这一处**：
    //   通知上带着「撤回」，是反悔的第二条路（应用内撤销条只活 3 秒），不能立刻撤；
    //   但也不能永不撤——待办已归档，通知还挂着「该办了」，点「完成」还会静默失败。
    //   - 应用内点圆圈 / 右滑、小组件圆圈：10 秒内反悔（unarchive）会取消这个定时器；
    //   - 小组件那条路 0.9.3 漏接过：通知永不撤，且重复待办再点一次通知「完成」
    //     会二次顺延、直接跳过一个周期（窗口从"永远"缩到 10 秒）；
    //   - 通知栏「完成」刻意**不撤**（done 态常驻才能随时撤回），由
    //     [completeFromNotification] 重发 done 后调 cancelScheduled 把自己摘出来。
    NotificationDismissScheduler.scheduleCancel(context, id)
    return true
}

/** Advance a repeating todo to its next occurrence and re-schedule its reminders. */
private suspend fun rollForward(todo: Todo) {
    val zone = AppTime.ZONE
    val due = todo.dueAtMillis
```

```
        if (due == null) { // no-date repeat: just bounce completedAt, keep it active
            dao.update(todo.copy(completedAt = System.currentTimeMillis()))
            return
        }
        val inst = java.time.Instant.ofEpochMilli(due)
        val curDate = inst.atZone(zone).toLocalDate()
        val curTime = inst.atZone(zone).toLocalTime()
        val today = LocalDate.now(zone)
        val nowTime = LocalTime.now(zone)
        var next = curDate
        var guard = 0
        // Advance by at least one step; keep advancing while the resulting date+time is not
        // in the future. Comparing only the date would leave "每天 18 点" completed at 22:00
        // pointing at today 18:00 again (already past -> immediate re-nag).
        do {
            next = next.plusDays(todo.repeatDays.toLong())
            guard++
        // guard 400→4000（P3-5）：走满后若仍在过去，会排一个过去时刻的闹钟（立即触发）。
        // 4000 天 ≈ 11 年，足够覆盖"待办 due 陈年老旧"的真实场景，循环本身是廉价的天数加法。
        } while ((next.isBefore(today) || (next == today && curTime.isBefore(nowTime))) &&
guard < 4000)
        val nextEpoch = ZonedDateTime.of(next, curTime, zone).toInstant().toEpochMilli()
        // 顺延时开始日同步平移（重复任务通常没有开始日，这里是防御性处理）。
        val nextStart = todo.startAtMillis?.let { it + (nextEpoch - due) }
        // 结束点同距平移（#1）：重复 + 时间区间（「每天下午 3 点到 5 点」编辑保存后才可能出现）
        // 只平移 due 会留下 endAtMillis < dueAtMillis 的坏数据，显示成「17:00 ~ 15:00」。
        val nextEnd = todo.endAtMillis?.let { it + (nextEpoch - due) }
        dao.updateContent(
            id = todo.id,
            title = todo.title,
            detail = todo.detail,
            dueAtMillis = nextEpoch,
            startAtMillis = nextStart,
            endAtMillis = nextEnd,
            hasExactTime = todo.hasExactTime,
            repeatDays = todo.repeatDays
        )
        ReminderScheduler.scheduleBlocking(
            context, todo.id, nextEpoch, todo.hasExactTime, nextStart, nextEnd
        )
    }

    suspend fun unarchive(id: Long) {
        // 待办回到活跃状态：撤掉归档时排的「10 秒后撤通知」（[archive] 里那一个）。
        // 收在 repository 里，和 archive 成对 —— 调用方（应用内撤销条 / 归档页恢复 /
        // 通知栏撤回）不用各自记得 cancel。重复任务的撤回不走这里（见
        // [undoFromNotification]），那条路由 NotificationActionReceiver 自己摘定时器。
        NotificationDismissScheduler.cancelScheduled(id)
        dao.getById(id)?.let {
            dao.update(it.copy(archived = false, completedAt = null))
            it.dueAtMillis?.let { due ->
```

```
                ReminderScheduler.scheduleFor(
                    context, id, due, it.hasExactTime, it.startAtMillis, it.endAtMillis
                )
            }
        }
    }

    /**
     * 删除（硬删除，不可恢复）。
     *
     * 入口在归档页，点「删除」会先弹确认对话框 —— 确认只有一道，但必须有：
     * 这条待办已经完成过，用户很容易把"清掉一条旧记录"和"就这么留着"看混。
     *
     * 闹钟和通知要**立刻**撤：待办已经从表里消失，留着闹钟只会在几分钟后弹出一条
     * 点进去找不到东西的提醒。延时撤通知那套（[NotificationDismissScheduler]）是给
     * "完成"用的 —— 删除没有反悔窗口，不需要留。
     */
    suspend fun delete(id: Long) {
        NotificationDismissScheduler.cancelScheduled(id)
        NotificationHelper.cancelReminder(context, id)
        ReminderScheduler.cancelForId(context, id)
        dao.deleteById(id)
    }

    /** Replace a todo's content (title / due / exact / repeat) from a fresh parse. */
    suspend fun editFromParsed(id: Long, parsed: ParsedTodo) {
        val due = parsedToDue(parsed)
        val start = parsedToStart(parsed)
        val end = parsedToEnd(parsed)
        dao.updateContent(
            id = id,
            title = parsed.cleanTitle,
            detail = parsed.detail.orEmpty(),
            dueAtMillis = due?.first,
            startAtMillis = start,
            endAtMillis = end,
            hasExactTime = parsed.hasExactTime,
            repeatDays = parsed.repeatDays
        )
        // Rebuild reminders for the new due time.
        if (due != null) {
            val archivedTodo = dao.getById(id)
            if (archivedTodo?.archived == false) {
                ReminderScheduler.scheduleBlocking(context, id, due.first, due.second, start,
end)
            }
        } else {
            ReminderScheduler.cancelForId(context, id)
        }
    }
```

```
suspend fun getById(id: Long): Todo? = dao.getById(id)

// ---------- notification action buttons (「完成」/「撤回」) ----------

/**
 * Notification 「完成」action. Intentionally delegates to [archive] so the notification
 * button and the list circle can never drift apart: normal todos are archived, repeating
 * ones roll forward to their next occurrence.
 */
suspend fun completeFromNotification(id: Long) {
    archive(id)
    // 通知栏「完成」刻意**不**撤通知：done 态带着「撤回」按钮，是这条路自己的反悔入口
    // （与应用内的 10 秒窗口不同，这里要能随时撤回）。把 archive 里排的延时撤通知
    // 摘掉，否则 10 秒后这条 done 通知会被自己人撤掉。
    NotificationDismissScheduler.cancelScheduled(id)
}
/**
 * Notification 「撤回」action — the inverse of [completeFromNotification].
 *
 * - Repeating todo (rolled forward by [rollForward]): step back exactly one period and
 *   re-arm its reminders. The todo may land in the past, which is correct: the user is
 *   saying "I did not actually finish this one".
 * - Normal todo: un-archive it and rebuild its reminders.
 *
 * @return the updated row, or null when the todo no longer exists.
 */
suspend fun undoFromNotification(id: Long): Todo? {
    val todo = dao.getById(id) ?: return null
    if (todo.repeatDays > 0 && SettingsRepo.autoRepeatEnabled(context)) {
        val cur = todo.dueAtMillis
        val prev = ReminderEngine.previousOccurrence(cur, todo.repeatDays)
        if (prev != null && cur != null) {
            // 开始日必须跟 due 回退**同样的幅度**——rollForward 顺延时平移的就是
            // `nextEpoch - due`（见 [rollForward]）。只回退 due 的话，重复+时间段
            // 的待办撤回后会留下 start > due：列表显示「10 月 27 日–26 日」这种倒挂文案，
            // isInProgress 恒 false，START 提醒也不再排（start >= due 被跳过）。
            // 重复+区间只能从「编辑一条重复待办 + 预览卡『+ 时间段』」造出来——
            // 解析器一句里不支持（Parser.stripRange 有拦截），所以这条路径很窄，
            // 但一旦踩到就是坏数据。
            val prevStart = todo.startAtMillis?.let { it - (cur - prev) }
            // 结束点回退同样幅度（#1），理由同 rollForward：不能留下 end < due 的倒挂。
            val prevEnd = todo.endAtMillis?.let { it - (cur - prev) }
            dao.updateContent(
                id = todo.id,
                title = todo.title,
                detail = todo.detail,
                dueAtMillis = prev,
                startAtMillis = prevStart,
                endAtMillis = prevEnd,
                hasExactTime = todo.hasExactTime,
```

```
                    repeatDays = todo.repeatDays
                )
                ReminderScheduler.scheduleBlocking(
                    context, todo.id, prev, todo.hasExactTime, prevStart, prevEnd
                )
            }
            // No-date repeat: there is no period to roll back, nothing to do.
            return dao.getById(id)
        }
        if (todo.archived) unarchive(id)
        return dao.getById(id)
    }
}
```

app/src/main/kotlin/com/lirui/oneline/data/model/FocusSession.kt

```
package com.lirui.oneline.data.model

import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * 一次已经结束的专注记录。
 *
 * ★ 为什么单独一张表，而不是往 `todos` 里加一列累计时长：
 *   专注功能是**完全独立**的（不绑定任何待办，见 DESIGN.md「专注」一节），
 *   把时长挂到待办上会带来两个死结 —— 删掉待办连历史时长一起没了；
 *   同一件事专注三次也不知道该记在哪一条上。历史是历史，待办是待办。
 *
 * ★ 存 `startedAtMillis` 而不是只存日期：跨午夜的那次专注（23:50 → 00:20）
 *   必须整段算在**开始那天**，否则热力图和「今日时长」会在午夜前后对不上。
 *   统计侧统一按 [com.lirui.oneline.util.AppTime.ZONE] 折算成日期，见 FocusStats。
 *
 * @param label          专注时填的说明，空串 = 没写（UI 显示「专注」）
 * @param plannedSeconds 开始前选的计划时长（25 分钟 = 1500）
 * @param actualSeconds  真正走了多久。走完 = plannedSeconds；
 *                       提前结束 = 实际经过的秒数（≥ 取消阈值才落库，见 FocusRules）
 * @param completed      是否自然走完。false = 中途结束
 */
@Entity(tableName = "focus_sessions")
data class FocusSession(
    @PrimaryKey(autoGenerate = true) val id: Long = 0L,
    val label: String = "",
    val startedAtMillis: Long,
    val endedAtMillis: Long,
    val plannedSeconds: Int,
    val actualSeconds: Int,
    val completed: Boolean,
    val createdAt: Long
)
```

app/src/main/kotlin/com/lirui/oneline/data/model/Todo.kt

```
package com.lirui.oneline.data.model
```

```
import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * A single reminder todo item.
 *
 * - [title]: the parsed description (time/date tokens removed).
 * - [detail]: optional free-text detail typed after the event title (everything after the
 *   first space of the remainder), e.g. "组会 要带笔记本去" -> title 组会, detail 要带笔记本去.
 * - [dueAtMillis]: the concrete due timestamp (in the device's timezone), or null for a
 *   "no-date" item that never fires a reminder.
 * - [hasExactTime]: whether the user gave an exact minute time (e.g. "15:30" or "下午 3 点").
 *   Drives the extra T-1h reminder.
 * - [repeatDays]: repeat step in days when completed. null/0 = no repeat.
 *   1 = daily, 7 = weekly, 30 = monthly (approx). On completion such a todo rolls forward
 *   to its next occurrence instead of being archived.
 * - [archived]: completed tasks are archived (hidden from the active list).
 * - [deletedAt]: **目前未使用**。0.9.3 一度把删除做成"回收站"（软删除 + 30 天保留），
 *   zuige 拍板不要—删除只要一次二次确认就够了。列和 DB 版本号保留在这里，
 *   是为了避免已经装过那一版 0.9.3 的设备发生**数据库降级**（Room 会直接抛异常）。
 *   代码里没有任何地方写它，恒为 null。
 */
@Entity(tableName = "todos")
data class Todo(
    @PrimaryKey(autoGenerate = true) val id: Long = 0L,
    val title: String,
    @ColumnInfo(defaultValue = "") val detail: String = "",
    val dueAtMillis: Long? = null,
    /**
     * 时间段的开始日（epoch 毫秒，按 AppTime.ZONE 落在当天）。
     * null = 单日待办，行为与旧版完全一致；非 null 时 [dueAtMillis] 的语义是「最后一天」，
     * 逾期判定仍只看 [dueAtMillis]（见 Format.isOverdue）。
     */
    val startAtMillis: Long? = null,
    /**
     * 时间区间的结束时刻（epoch 毫秒，与 [dueAtMillis] 同一天）。
     * null = 普通待办；非 null 时 [dueAtMillis] 是开始点（提醒在这响），本字段是结束点，
     * 用于列表显示「15:00 ~ 17:00」与逾期判定（过了结束点才算逾期，进行中不标红）。
     */
    @ColumnInfo(defaultValue = "NULL") val endAtMillis: Long? = null,
    val hasExactTime: Boolean = false,

    @ColumnInfo(defaultValue = "0") val repeatDays: Int = 0,
    @ColumnInfo(defaultValue = "0") val archived: Boolean = false,
    @ColumnInfo(defaultValue = "0") val createdAt: Long = System.currentTimeMillis(),
    @ColumnInfo(defaultValue = "0") val completedAt: Long? = null,
    /** 见类注释：未使用的历史列，只为保住 DB 版本号。 */
    @ColumnInfo(defaultValue = "NULL") val deletedAt: Long? = null
```

```
)
```

app/src/main/kotlin/com/lirui/oneline/focus/FocusActionReceiver.kt

```
package com.lirui.oneline.focus

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.launch

/**
 * 通知条上的「暂停 / 继续 / 结束」。
 *
 * ★ 「结束」在这里**不问**"这么短要不要记"：通知栏里弹不了对话框，
 *   而且从通知点结束本来就是"我要收手"的意思。口径定为：
 *   够长（≥ [FocusRules.ASK_THRESHOLD_SECONDS]）就记，太短就当没发生 ——
 *   和 UI 里"太短会问"并不冲突，因为那是**用户看着计时器**做的决定。
 */
class FocusActionReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        val appContext = context.applicationContext
        val now = System.currentTimeMillis()
        when (intent.action) {
            ACTION_PAUSE -> FocusEngine.pause(appContext, now)
            ACTION_RESUME -> FocusEngine.resume(appContext, now)
            ACTION_FINISH -> {
                val pending = goAsync()
                CoroutineScope(SupervisorJob() + Dispatchers.IO).launch {
                    try {
                        val run = FocusStore.active(appContext)
                        if (run != null) {
                            val elapsed = FocusRules.elapsedSeconds(run, now)
                            FocusEngine.finishEarly(
                                appContext,
                                now,
                                record = FocusRules.shouldRecordSilently(elapsed)
                            )
                        }
                    } finally {
                        pending.finish()
                    }
                }
            }
        }
    }

    companion object {
```

```
        const val ACTION_PAUSE = "com.lirui.oneline.action.FOCUS_PAUSE"
        const val ACTION_RESUME = "com.lirui.oneline.action.FOCUS_RESUME"
        const val ACTION_FINISH = "com.lirui.oneline.action.FOCUS_FINISH"
    }
}
```

app/src/main/kotlin/com/lirui/oneline/focus/FocusAlarmReceiver.kt

```
package com.lirui.oneline.focus

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.launch

/**
 * 专注时间到。由 [FocusScheduler] 排的精确闹钟拉起。
 *
 * 用 `goAsync()` 包住写库：`onReceive` 一返回进程优先级立刻回落，
 * 协程可能还没 insert 完就被回收 —— 表现就是"到点了、通知也变了，但记录里没有"。
 */
class FocusAlarmReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action != FocusScheduler.ACTION_FOCUS_DONE) return
        val appContext = context.applicationContext
        val pending = goAsync()
        CoroutineScope(SupervisorJob() + Dispatchers.IO).launch {
            try {
                FocusEngine.complete(appContext)
            } finally {
                pending.finish()
            }
        }
    }
}
```

app/src/main/kotlin/com/lirui/oneline/focus/FocusEngine.kt

```
package com.lirui.oneline.focus

import android.content.Context
import com.lirui.oneline.data.FocusRepository

/**
 * 专注的**唯一状态机**。
 *
 * 所有入口（专注页的按钮、通知上的暂停/结束、到点闹钟、开机重建）都走这里，
 * 不在各自的地方再写一遍"存盘 + 发通知 + 排闹钟"这三件套 ——
 * 少写一处就会漏一处，最后表现成"某条路走完不落库"。
 */
```

```
object FocusEngine {

    /** 开一次专注。返回新建的 run（UI 直接拿去渲染）。 */
    fun start(
        context: Context,
        minutes: Int,
        label: String,
        nowMillis: Long,
        countUp: Boolean = false
    ): FocusRun {
        val run = FocusRun(
            // 正计时没有目标时长：plannedSeconds = 0，也就不会有"到点"这件事。
            plannedSeconds = if (countUp) 0 else minutes * 60,
            accumulatedMillis = 0L,
            segmentStartMillis = nowMillis,
            paused = false,
            startedAtMillis = nowMillis,
            label = label.trim(),
            countUp = countUp
        )
        FocusStore.saveActive(context, run)
        FocusStore.setLastMinutes(context, minutes)
        FocusStore.setLastCountUp(context, countUp)
        FocusNotifier.showOngoing(context, run, nowMillis)
        // 正计时不排闹钟：它没有终点，只能手动结束。
        if (!countUp) FocusScheduler.schedule(context, nowMillis + run.plannedMillis)
        return run
    }

    fun pause(context: Context, nowMillis: Long): FocusRun? {
        val run = FocusStore.active(context) ?: return null
        val paused = run.pause(nowMillis)
        FocusStore.saveActive(context, paused)
        // 暂停了就别再响：闹钟留着会在暂停期间把这次专注"判完成"。
        FocusScheduler.cancel(context)
        FocusNotifier.showOngoing(context, paused, nowMillis)
        return paused
    }

    fun resume(context: Context, nowMillis: Long): FocusRun? {
        val run = FocusStore.active(context) ?: return null
        val resumed = run.resume(nowMillis)
        FocusStore.saveActive(context, resumed)
        FocusNotifier.showOngoing(context, resumed, nowMillis)
        if (!resumed.countUp) {
            FocusScheduler.schedule(context, nowMillis + FocusRules.remainingMillis(resumed,
nowMillis))
        }
        return resumed
    }
```

```
    /**
     * 自然走完。
     *
     * @return true = 这次真的落库了；false = 已经没有进行中的专注（被另一条路先收走了）。
     *         两条触发源（闹钟 / 前台 tick）靠 [FocusStore.takeActive] 的原子性去重。
     */
    suspend fun complete(context: Context, nowMillis: Long = System.currentTimeMillis()):
Boolean {
        val run = FocusStore.takeActive(context) ?: return false
        FocusScheduler.cancel(context)
        FocusNotifier.cancelOngoing(context)
        // elapsed 已被 FocusRules 截到 planned，正常到点就是满时长；
        // 闹钟被系统拖晚几秒也不会记成"超时"。
        val seconds = FocusRules.elapsedSeconds(run, nowMillis).coerceAtLeast(1)
        FocusRepository(context).record(
            label = run.label,
            startedAtMillis = run.startedAtMillis,
            endedAtMillis = nowMillis,
            plannedSeconds = run.plannedSeconds,
            actualSeconds = seconds,
            completed = true
        )
        FocusNotifier.showDone(context, seconds, run.label)
        return true
    }

    /**
     * 中途结束。
     *
     * @param record 用户的选择：太短时会先问一句（见 [FocusRules.shouldAskBeforeDiscard]），
     *               够长时调用方直接传 true 不问。传 false 表示"就当没发生过"。
     * @return 记入的秒数；0 表示什么都没记。
     */
    suspend fun finishEarly(context: Context, nowMillis: Long, record: Boolean): Int {
        val run = FocusStore.takeActive(context) ?: return 0
        FocusScheduler.cancel(context)
        FocusNotifier.cancelOngoing(context)
        if (!record) return 0
        val seconds = FocusRules.elapsedSeconds(run, nowMillis)
        if (seconds <= 0) return 0
        FocusRepository(context).record(
            label = run.label,
            startedAtMillis = run.startedAtMillis,
            endedAtMillis = nowMillis,
            plannedSeconds = run.plannedSeconds,
            actualSeconds = seconds,
            // ★ 正计时没有"提前"这个概念 —— 点结束就是正常结束，记成已完成。
            //   倒计时才有"没走完就停了"，记 completed=false（列表里显示「提前结束」）。
            completed = run.countUp
        )
        return seconds
```

```
    }

    /** 计时中途改名：只更新 run，不动闹钟（时长没变）。 */
    fun rename(context: Context, label: String, nowMillis: Long): FocusRun? {
        val run = FocusStore.active(context) ?: return null
        val renamed = run.rename(label.trim())
        FocusStore.saveActive(context, renamed)
        FocusNotifier.showOngoing(context, renamed, nowMillis)
        return renamed
    }

    /**
     * 冷启动 / 开机 / 升级后对账：闹钟条目已经没了，但 run 还在盘上。
     *
     * 这是"重启后专注还能继续"的唯一原因 —— AlarmManager 的条目不落盘，
     * 不重排的话计时会一直走到天荒地老也不会响。
     */
    fun restore(context: Context, nowMillis: Long) {
        val run = FocusStore.active(context) ?: return
        if (run.paused) {
            // 暂停中本来就不该有闹钟，只把通知补回来（进程重启后通知还在，但文案可能过期）。
            FocusNotifier.showOngoing(context, run, nowMillis)
            return
        }
        val remaining = FocusRules.remainingMillis(run, nowMillis)
        if (run.countUp || remaining <= 0L) {
            // 正计时本来就没有闹钟；倒计时已经过点了，交给 UI 的 tick 或闹钟去 complete。
            FocusNotifier.showOngoing(context, run, nowMillis)
            return
        }
        FocusScheduler.schedule(context, nowMillis + remaining)
        FocusNotifier.showOngoing(context, run, nowMillis)
    }
}
```

app/src/main/kotlin/com/lirui/oneline/focus/FocusNotifier.kt

```
package com.lirui.oneline.focus

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import androidx.core.app.NotificationCompat
import com.lirui.oneline.MainActivity
import com.lirui.oneline.R

/**
 * 专注相关的两条通知。
 *
```

```
 * ★ 为什么**不需要**前台服务（这一点和 `KeepAliveService` 完全不同）：
 *   倒计时交给系统的 chronometer 渲染 —— `setUsesChronometer(true)` +
 *   `setChronometerCountDown(true)` + `setWhen(结束时刻)`。系统每帧自己算剩余时间，
 *   **我们的进程死不死都不影响**它继续走。真正到点那一下由 AlarmManager 兜底
 *   （见 [FocusScheduler]），也是 system_server 持有，进程被回收照样响。
 *
 *   所以这里没有 `startForeground`、没有 `FOREGROUND_SERVICE` 权限、
 *   没有"后台服务被国产 ROM 杀掉"这类问题，也少一道商店审核材料。
 *   代价只有一条：暂停/结束时通知不能自己变，得靠 [FocusActionReceiver] 被点一下。
 */
object FocusNotifier {

    const val CHANNEL_ONGOING = "focus_ongoing"
    const val CHANNEL_DONE = "focus_done"

    const val NOTIF_ONGOING = 9101
    const val NOTIF_DONE = 9102

    const val ACTION_OPEN_FOCUS = "com.lirui.oneline.action.OPEN_FOCUS"

    fun init(context: Context) {
        val nm = context.getSystemService(NotificationManager::class.java) ?: return
        // 常驻条：静音、无横幅、不震动 —— 它只是"还剩多久"的显示板，不是提醒。
        nm.createNotificationChannel(
            NotificationChannel(CHANNEL_ONGOING, "专注倒计时",
NotificationManager.IMPORTANCE_LOW).apply {
                description = "专注进行中的常驻倒计时"
                setShowBadge(false)
                setSound(null, null)
                enableVibration(false)
            }
        )
        // 完成条：要响、要横幅。它是一次真正的事件，用户应该被打断。
        nm.createNotificationChannel(
            NotificationChannel(CHANNEL_DONE, "专注完成",
NotificationManager.IMPORTANCE_HIGH).apply {
                description = "一次专注走完时的提醒"
                setShowBadge(true)
            }
        )
    }

    /**
     * 刷新常驻倒计时条。
     *
     * 运行中走系统 chronometer（不再自己每秒重发通知 —— 那是最耗电的做法）；
     * 暂停时改成静态文案，因为 chronometer 没有"停住"这个能力。
     */
    fun showOngoing(context: Context, run: FocusRun, nowMillis: Long) {
        val nm = context.getSystemService(NotificationManager::class.java) ?: return
        val elapsedSec = (FocusRules.elapsedMillis(run, nowMillis) / 1000L).toInt()
        val remainingSec = (FocusRules.remainingMillis(run, nowMillis) / 1000L).toInt()
```

```
        val title = run.label.ifBlank { if (run.countUp) "正计时中" else "专注中" }

        val builder = NotificationCompat.Builder(context, CHANNEL_ONGOING)
            .setSmallIcon(R.drawable.ic_focus_notification)
            .setContentTitle(title)
            .setOngoing(true)
            .setOnlyAlertOnce(true)
            .setSilent(true)
            .setCategory(NotificationCompat.CATEGORY_PROGRESS)
            .setPriority(NotificationCompat.PRIORITY_LOW)
            .setContentIntent(openFocusIntent(context))
            .addAction(0, if (run.paused) "继续" else "暂停", actionIntent(context, if
(run.paused) FocusActionReceiver.ACTION_RESUME else FocusActionReceiver.ACTION_PAUSE))
            .addAction(0, "结束", actionIntent(context, FocusActionReceiver.ACTION_FINISH))

        if (run.paused) {
            val pausedText = if (run.countUp) {
                "已暂停 · 已专注 ${FocusRules.formatClock(elapsedSec)}"
            } else {
                "已暂停 · 剩余 ${FocusRules.formatClock(remainingSec)}"
            }
            builder.setContentText(pausedText)
                .setShowWhen(false)
        } else {
            builder.setUsesChronometer(true)
                .setShowWhen(true)
            if (run.countUp) {
                // 正计时：chronometer 往上数。setWhen 要给"起点"，
                // 也就是现在往前推已经过去的那段（含暂停前累计的）。
                builder.setChronometerCountDown(false)
                    .setContentText("已专注")
                    .setWhen(nowMillis - FocusRules.elapsedMillis(run, nowMillis))
            } else {
                // 倒计时：chronometer 往下数，setWhen 是终点。
                builder.setChronometerCountDown(true)
                    .setContentText("剩余时间")
                    .setWhen(nowMillis + FocusRules.remainingMillis(run, nowMillis))
            }
        }

        runCatching { nm.notify(NOTIF_ONGOING, builder.build()) }
    }

    fun cancelOngoing(context: Context) {
        val nm = context.getSystemService(NotificationManager::class.java) ?: return
        runCatching { nm.cancel(NOTIF_ONGOING) }
    }

    /** 走完了。文案刻意短：一句话说清"多久、什么"。 */
    fun showDone(context: Context, seconds: Int, label: String) {
        val nm = context.getSystemService(NotificationManager::class.java) ?: return
```

```
        val what = label.ifBlank { "专注" }
        val n = NotificationCompat.Builder(context, CHANNEL_DONE)
            .setSmallIcon(R.drawable.ic_focus_notification)
            .setContentTitle("专注完成")
            .setContentText("$what · ${FocusRules.formatDuration(seconds)}")
            .setAutoCancel(true)
            .setCategory(NotificationCompat.CATEGORY_STATUS)
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setDefaults(Notification.DEFAULT_ALL)
            .setContentIntent(openFocusIntent(context))
            .build()
        runCatching { nm.notify(NOTIF_DONE, n) }
    }

    private fun openFocusIntent(context: Context): PendingIntent = PendingIntent.getActivity(
        context,
        0,
        Intent(context, MainActivity::class.java)
            .setAction(ACTION_OPEN_FOCUS)
            .addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP),
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
    )

    private fun actionIntent(context: Context, action: String): PendingIntent =
        PendingIntent.getBroadcast(
            context,
            action.hashCode(),
            Intent(context, FocusActionReceiver::class.java).setAction(action),
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
}
```

app/src/main/kotlin/com/lirui/oneline/focus/FocusRules.kt

```
package com.lirui.oneline.focus

/**
 * 一次正在进行的专注。
 *
 * ★ 为什么记 `segmentStartMillis` + `accumulatedMillis` 而不是每秒 `++`：
 *   计时必须**只依赖时间戳**。Compose 的 `LaunchedEffect { while(true){ delay(1000) } }`
 *   在息屏 / 后台 / 进程被回收时会被挂起或直接掐掉，靠它累加，回来一看少了几十分钟。
 *   现在的模型是"记下起点，需要时做差"——UI 每秒只是**重绘**，不负责计时。
 *   这也让暂停/继续变成纯数据操作（见 [paused]），不牵涉任何协程生命周期。
 *
 * @param plannedSeconds     计划时长（秒）。**正计时时为 0** —— 没有目标就没有终点
 * @param accumulatedMillis  之前各运行段累计的时间（暂停时把它吃进来）
 * @param segmentStartMillis 当前运行段的起点。**暂停时这个值没有意义**，不要读
 * @param paused             是否处于暂停
 * @param startedAtMillis    整次专注真正的开始时刻（统计按它归日）
 * @param label              专注时填的说明，空串 = 没写
 * @param countUp            正计时（从 0 往上数）vs 倒计时（从 planned 往下数）
```

```
 */
data class FocusRun(
    val plannedSeconds: Int,
    val accumulatedMillis: Long,
    val segmentStartMillis: Long,
    val paused: Boolean,
    val startedAtMillis: Long,
    val label: String = "",
    val countUp: Boolean = false
) {

    val plannedMillis: Long get() = plannedSeconds * 1000L

    /** 暂停：把当前运行段折算进累计，之后 elapsed 不再随墙钟增长。 */
    fun pause(nowMillis: Long): FocusRun {
        if (paused) return this
        val delta = (nowMillis - segmentStartMillis).coerceAtLeast(0L)
        return copy(accumulatedMillis = accumulatedMillis + delta, paused = true)
    }

    /** 继续：开一个新的运行段，累计值保持不变。 */
    fun resume(nowMillis: Long): FocusRun {
        if (!paused) return this
        return copy(segmentStartMillis = nowMillis, paused = false)
    }

    /** 不改时长、只换说明（计时中途改名字用）。 */
    fun rename(label: String): FocusRun = copy(label = label)
}

object FocusRules {

    /**
     * 中途结束时「够不够长」的分界线（秒）。
     *
     * 语义（zuige 定的口径）：
     *  - **≥ 60 秒** → 这是真干了一会儿，直接记入统计（`completed = false`）；
     *  - **< 60 秒** → 分不清是误触还是真开始了，弹一句问用户记不记。
     *
     * 之所以不是"短的一律丢弃"：30 秒也可能是真的开了个头，
     * 用户自己最清楚；而 40 分钟的提前结束绝不该因为"没走完"就白干。
     */
    const val ASK_THRESHOLD_SECONDS = 60

    /** 时长预设（分钟）。 */
    val PRESET_MINUTES: List<Int> = listOf(15, 25, 45, 60)

    /** 首次进入默认 25 分钟（一个标准番茄钟）。 */
    const val DEFAULT_MINUTES = 25
```

```
    /**
     * 已经过的毫秒。暂停时不再增长。
     *
     * ★ 倒计时会截到 planned（闹钟晚几秒不算"超时"）；**正计时不截** ——
     *   正计时本来就没有上限，截了就等于永远停在 0。
     */
    fun elapsedMillis(run: FocusRun, nowMillis: Long): Long {
        val running = if (run.paused) 0L else (nowMillis -
run.segmentStartMillis).coerceAtLeast(0L)
        val total = run.accumulatedMillis + running
        return if (run.countUp) total else total.coerceAtMost(run.plannedMillis)
    }

    /** 已经过的整秒（向下取整 —— 显示 00:01 的时候不能真的已经过了 2 秒）。 */
    fun elapsedSeconds(run: FocusRun, nowMillis: Long): Int =
        (elapsedMillis(run, nowMillis) / 1000L).toInt()

    /** 剩余毫秒，不为负。正计时恒为 0（没有终点）。 */
    fun remainingMillis(run: FocusRun, nowMillis: Long): Long =
        if (run.countUp) 0L
        else (run.plannedMillis - elapsedMillis(run, nowMillis)).coerceAtLeast(0L)

    /**
     * 剩余比例 1f → 0f，圆环按它画。
     *
     * 正计时恒为 1f：它没有"剩余"可言，环就整圈亮着当装饰 ——
     * 硬凑一个进度（比如按 60 分钟算百分比）是在编造用户没设定的目标。
     */
    fun remainingFraction(run: FocusRun, nowMillis: Long): Float {
        if (run.countUp) return 1f
        if (run.plannedMillis <= 0L) return 0f
        return (remainingMillis(run, nowMillis).toFloat() / run.plannedMillis).coerceIn(0f, 1f)
    }

    /** 时间到没到。正计时永远不到（只能手动结束）。 */
    fun isFinished(run: FocusRun, nowMillis: Long): Boolean =
        !run.countUp && elapsedMillis(run, nowMillis) >= run.plannedMillis

    /** 中途结束且够长 → 不问，直接记。 */
    fun shouldRecordSilently(elapsedSeconds: Int): Boolean = elapsedSeconds >=
ASK_THRESHOLD_SECONDS

    /** 中途结束但太短 → 问一句「要记下来吗」。0 秒不问（什么都没发生）。 */
    fun shouldAskBeforeDiscard(elapsedSeconds: Int): Boolean =
        elapsedSeconds in 1 until ASK_THRESHOLD_SECONDS

    /**
     * 倒计时显示。`25:00` / `01:05:00`。
     *
     * 一小时以内只给 mm:ss —— 专注场景里 45 分钟是常态，多一个 `00:` 前缀
     * 会让圆环中心的字变小，反而不如两位数字醒目。
     */
```

```
    fun formatClock(totalSeconds: Int): String {
        val s = totalSeconds.coerceAtLeast(0)
        val h = s / 3600
        val m = (s % 3600) / 60
        val sec = s % 60
        return if (h > 0) {
            String.format(java.util.Locale.US, "%d:%02d:%02d", h, m, sec)
        } else {
            String.format(java.util.Locale.US, "%02d:%02d", m, sec)
        }
    }

    /**
     * 时长的人话写法：`2 小时 15 分` / `45 分钟` / `38 秒`。
     *
     * 统计卡片上不用 `2h15m`：这个应用的其余文案全是中文（「明天」「归档」），
     * 混进英文单位会显得是另一套设计语言。
     */
    fun formatDuration(totalSeconds: Int): String {
        val s = totalSeconds.coerceAtLeast(0)
        if (s < 60) return "${s}秒"
        val h = s / 3600
        val m = (s % 3600) / 60
        return when {
            h == 0 -> "${m}分钟"
            m == 0 -> "${h}小时"
            else -> "${h}小时${m}分"
        }
    }
}
```

app/src/main/kotlin/com/lirui/oneline/focus/FocusScheduler.kt

```
package com.lirui.oneline.focus

import android.app.AlarmManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.os.Build

/**
 * 专注到点的闹钟。
 *
 * ★ 为什么必须用 AlarmManager，而不是 `Handler.postDelayed` / 协程 `delay`：
 *   后两者都活在**本进程**里。专注 25 分钟，中间被系统回收、被一键清理、被划掉，
 *   定时器就没了 —— 用户回来发现"计时停了、通知还在走"，而这一次专注永远不落库。
 *   闹钟由 system_server 持有，进程死了照样到点拉起 [FocusAlarmReceiver]。
 *
 * 只用一个 requestCode：同一时刻只可能有**一次**专注在跑，
 * 再开第二次之前必须先结束第一次（UI 也不给这个入口）。
 */
```

（以上为源程序前连续 30 页，每页 50 行）

（以下为源程序后连续 30 页，每页 50 行）

源程序共 46 个文件、13088 行，此处提交前 1500 行与后 1500 行

```
     *  [showWeekday]=false 时省略「周几」（小组件默认不带星期，zuige 要求）。
     *  [showTime]=false 时只到日期（「10 月 23 日」），不带「14 点」——小组件「只显示日期」开关用。
     *  参数顺序保持 today 在 showTime 之前，老的 `dueLine(due, today)` 位置调用不受影响。 */
    fun dueLine(
        due: Long,
        showWeekday: Boolean = true,
        today: LocalDate = AppTime.today(),
        showTime: Boolean = true
    ): String {
        val zdt = java.time.Instant.ofEpochMilli(due).atZone(AppTime.ZONE)
        val d = zdt.toLocalDate()
        val t = zdt.toLocalTime()
        val parts = ArrayList<String>()
        if (d.year != today.year) parts += "%d 年".format(d.year, Locale.CHINA)
        parts += "%d 月%d 日".format(Locale.CHINA, d.monthValue, d.dayOfMonth)
        if (showWeekday) parts += WEEKDAYS_CN[d.dayOfWeek.value - 1]
        if (showTime) {
            parts += if (t.minute == 0) "${t.hour}点" else String.format("%d:%02d", t.hour,
t.minute)
        }
        return parts.joinToString(" ")
    }

    /**
     * Single source of truth for "is this todo overdue?" — the list grouping and the red
     * marking MUST both call this, otherwise you get rows sitting under 已逾期 that aren't
     * red (or vice versa).
     *
     * Two different clocks are involved:
     * - **exact time given** ("23:39"): overdue once that instant has passed.
     * - **date only** ("今天开会"): the whole day is still available, so it is only overdue
     *   once the *date* has passed. (The row still carries the default remind time of 09:00
     *   for notification purposes, but falling back to it for the overdue test would
     *   make every date-only todo instantly red as soon as 09:00 has gone by.)
     *
     * `overduePin` (设置 → 逾期置顶) deliberately does NOT gate this: it only controls
     * sorting. Colouring must never depend on a sort preference.
     */
    fun isOverdue(
        due: Long?,
        today: LocalDate = AppTime.today(),
        nowMillis: Long = AppTime.nowMillis(),
        hasExactTime: Boolean = true
    ): Boolean {
        if (due == null) return false
        if (!hasExactTime) return dueAtMillis(due).isBefore(today)
        return due <= nowMillis
    }

    // ---------- 时间段（v0.8.4）----------
    //
```

```
    // 约定：start = 开始日（可空，null = 单日），end = 最后一天（即 dueAtMillis）。
    // 逾期永远只看 end —— 见 [isOverdue]，所以「最后一天结束才算逾期」是天然成立的。

    /** 今天是否落在 [start, end] 之间（含两端）。单日（start == null）恒 false。 */
    fun isInProgress(
        start: Long?,
        end: Long?,
        today: LocalDate = AppTime.today()
    ): Boolean {
        if (start == null || end == null) return false
        val s = dueAtMillis(start)
        val e = dueAtMillis(end)
        return !today.isBefore(s) && !today.isAfter(e)
    }

    /** 距结束日还有几天：0 = 今天结束，负数 = 已过结束日。 */
    fun daysLeft(end: Long, today: LocalDate = AppTime.today()): Long =
        java.time.temporal.ChronoUnit.DAYS.between(today, dueAtMillis(end))

    /**
     * 时间段的一行文案：`10 月 21 日–23 日` / `10 月 21 日–11 月 3 日` / `2026 年 10 月 21 日–2027 年 1 月
3 日`，
     * 末尾按需带星期与结束时刻。单日退回 [dueLine]。
     *
     * [endTimeMillis] 非空（#1 时间区间：「周一到周三 下午 3 点到 5 点」）= 末尾渲染成
     * 「15 点~17 点」而不是只带结束时刻 —— 日期段 + 时间区间的组合只有编辑一条重复待办
     * 才造得出来，但一旦出现就得显示对。
     */
    fun rangeLine(
        start: Long?,
        end: Long,
        showWeekday: Boolean = true,
        today: LocalDate = AppTime.today(),
        showTime: Boolean = true,
        endTimeMillis: Long? = null
    ): String {
        if (start == null) return dueLine(end, showWeekday, today, showTime)
        val s = dueAtMillis(start)
        val e = dueAtMillis(end)
        if (s == e) return dueLine(end, showWeekday, today, showTime)

        val suffix = buildString {
            if (showWeekday) append(" ").append(WEEKDAYS_CN[e.dayOfWeek.value - 1])
            if (showTime) {
                val t = java.time.Instant.ofEpochMilli(end).atZone(AppTime.ZONE).toLocalTime()
                if (endTimeMillis != null) {
                    val te =
java.time.Instant.ofEpochMilli(endTimeMillis).atZone(AppTime.ZONE).toLocalTime()
                    append(" ").append(
                        "%s~%s".format(
                            if (t.minute == 0) "${t.hour}点" else String.format("%d:%02d",
t.hour, t.minute),
                            if (te.minute == 0) "${te.hour}点" else String.format("%d:%02d",
te.hour, te.minute)
```

```
                        )
                    )
                } else {
                    append(" ").append(if (t.minute == 0) "${t.hour}点" else
String.format("%d:%02d", t.hour, t.minute))
                }
            }
        }
        return when {
            s.year == e.year && s.monthValue == e.monthValue ->
                "%d 月%d 日–%d 日%s".format(Locale.CHINA, s.monthValue, s.dayOfMonth,
e.dayOfMonth, suffix)
            s.year == e.year ->
                "%d 月%d 日–%d 月%d 日%s".format(Locale.CHINA, s.monthValue, s.dayOfMonth,
e.monthValue, e.dayOfMonth, suffix)
            else ->
                "%d 年%d 月%d 日–%d 年%d 月%d 日%s".format(Locale.CHINA, s.year, s.monthValue,
s.dayOfMonth, e.year, e.monthValue, e.dayOfMonth, suffix)
        }
    }

    /**
     * 能被解析器**原样读回去**的区间句子：`10 月 24 日到 26 日` / `10 月 28 日到 11 月 3 日` / 跨年带年份。
     * 编辑待办时用它回填输入框 —— 不能用 [monthDayRange]（`21–26 日` 解析器不认识）。
     */
    fun rangeSentence(start: Long, end: Long): String {
        val s = dueAtMillis(start)
        val e = dueAtMillis(end)
        if (s == e) return monthDay(e)
        return when {
            s.year == e.year && s.monthValue == e.monthValue ->
                "%d 月%d 日到%d 日".format(Locale.CHINA, s.monthValue, s.dayOfMonth, e.dayOfMonth)
            s.year == e.year ->
                "%d 月%d 日到%d 月%d 日".format(
                    Locale.CHINA, s.monthValue, s.dayOfMonth, e.monthValue, e.dayOfMonth
                )
            else ->
                "%d 年%d 月%d 日到%d 年%d 月%d 日".format(
                    Locale.CHINA, s.year, s.monthValue, s.dayOfMonth, e.year, e.monthValue,
e.dayOfMonth
                )
        }
    }

    /**
     * 编辑待办时回填输入框的句子。**必须能被 [com.lirui.oneline.parser.Parser] 原样读回成同一条待办
**：
     * - 时间段写成 [rangeSentence]（`10 月 24 日到 26 日`），否则开始日会丢、区间退化成单日；
     * - 单日只写日期，**不拼星期**。旧实现拼出「10 月 26 日 周一」，而「周一」会被解析成
     *   本周一（10/19），一点保存就多出一条已经过期的待办。
     *
     * [endAtMillis] 非空（#1 时间区间）时，时刻写成「15 点到 17 点」而不是「15 点」——
     * 否则一保存，结束点就丢了（区间退化成普通待办）。用 24 小时制裸数字，不拼「下午」：
     * 「下午 3 点到 5 点」与「15 点到 17 点」解析结果一致，后者没有时段词歧义。
     */
    fun editSentence(
```

```
        title: String,
        detail: String,
        start: Long?,
        due: Long?,
        hasExactTime: Boolean = true,
        today: LocalDate = AppTime.today(),
        endAtMillis: Long? = null
    ): String = buildString {
        append(title)
        if (detail.isNotBlank()) {
            append(" ")
            append(detail)
        }
        if (due == null) return@buildString
        append(" ")
        if (start != null) {
            append(rangeSentence(start, due))
        } else {
            val d = dueAtMillis(due)
            append(if (d == today) "今天" else monthDay(d))
        }
        // ★ 只写日期的待办**不能**回填时刻：句子里的「9 点」会被解析成 hasExactTime=true，
        //   于是「最后一天结束才算逾期」被悄悄改成「最后一天 9 点就变红」。
        if (!hasExactTime) return@buildString
        // Pinned zone —— 回填的句子必须和创建时读到的墙钟一致，不受设备时区影响。
        val t = java.time.Instant.ofEpochMilli(due).atZone(AppTime.ZONE).toLocalTime()
        append(" ")
        if (endAtMillis != null) {
            val te =
java.time.Instant.ofEpochMilli(endAtMillis).atZone(AppTime.ZONE).toLocalTime()
            append(timeWord(t)).append("到").append(timeWord(te))
        } else {
            append(timeWord(t))
        }
    }

    /** 「15 点」/「15 点 30 分」—— 回填输入框用，必须能被解析器原样读回（见 [editSentence]）。 */
    private fun timeWord(t: java.time.LocalTime): String =
        if (t.minute == 0) "${t.hour}点" else "${t.hour}点${t.minute}分"

    /** 右侧紧凑范围：`21–23 日` / `10/21–11/3`；单日退回 [monthDay]。 */    fun
monthDayRange(start: Long?, end: Long): String {
        if (start == null) return monthDay(dueAtMillis(end))
        val s = dueAtMillis(start)
        val e = dueAtMillis(end)
        if (s == e) return monthDay(e)
        return if (s.year == e.year && s.monthValue == e.monthValue) {
            "%d–%d 日".format(Locale.CHINA, s.dayOfMonth, e.dayOfMonth)
        } else {
            "%d/%d–%d/%d".format(Locale.CHINA, s.monthValue, s.dayOfMonth, e.monthValue,
e.dayOfMonth)
        }
    }
```

```
    /**
     * 时间段的状态短语：`逾期` / `最后一天` / `还剩 N 天`；单日（start == null）返回 null。
     *
     * 没有「未开始」：还没开始的时间段本来就该安静地待在「明天/本周」里，
     * 再挂一个"未开始"只是噪音（zuige 2026-10-23 明确要求去掉）。
     */
    fun rangeStatus(
        start: Long?,
        end: Long,
        today: LocalDate = AppTime.today(),
        nowMillis: Long = AppTime.nowMillis(),
        hasExactTime: Boolean = true
    ): String? {
        if (start == null) return null
        if (isOverdue(end, today, nowMillis, hasExactTime)) return "逾期"
        if (today.isBefore(dueAtMillis(start))) return null
        if (today == dueAtMillis(end)) return "最后一天"
        val left = daysLeft(end, today)
        return if (left > 0) "还剩${left}天" else null
    }
}
```

app/src/main/kotlin/com/lirui/oneline/util/HomeGrouping.kt

```
package com.lirui.oneline.util

import java.time.DayOfWeek
import java.time.LocalDate

/**
 * The three "boards" (板块) the home list is split into. Their order and visibility are user
 * settings (设置 → 首页分组), which is why this is an enum and not a hard-coded `when` chain.
 *
 * A dated todo that is already overdue belongs to [OVERDUE], never to [DATED] — the two are
 * mutually exclusive, otherwise one todo would show up twice.
 */
enum class HomeBoard(val id: String, val label: String) {
    OVERDUE("overdue", "逾期"),
    DATED("dated", "有日期"),
    UNDATED("undated", "无期");

    companion object {
        val all: List<HomeBoard> = entries

        /**
         * 默认顺序 = 逾期 → 无期 → 有日期（zuige 2026-10-23 定为首装默认）。
         *
         * 逾期置顶：该处理的先看见；有日期垫底，因为它们自己带时间、有单独的档位分组。
         */
        val defaultOrder: List<HomeBoard> = listOf(OVERDUE, UNDATED, DATED)
```

```
        fun byId(id: String): HomeBoard? = entries.firstOrNull { it.id == id }
    }
}

/**
 * Sub-groups *inside* [HomeBoard.DATED]. Which ones are actually rendered is user-selectable;
 * unselected buckets are merged into the next selected one (see [HomeGrouping.resolveBucket]).
 *
 * Declaration order matters: it is the chronological order of the timeline and the order the
 * merge walks in.
 */
enum class HomeBucket(val id: String, val label: String) {
    TODAY("today", "今日"),
    TOMORROW("tomorrow", "明天"),
    DAY_AFTER("day_after", "后天"),
    THIS_WEEK("this_week", "本周"),
    NEXT_WEEK("next_week", "下周"),
    LATER("later", "更晚");

    companion object {
        val all: List<HomeBucket> = entries

        /**
         * 默认只勾 今日 / 本周 / 更晚（zuige 2026-10-23 定为首装默认）。
         *
         * 明天 / 后天 / 下周不单独成组，按 [HomeGrouping.resolveBucket] 并入下一个勾中的档位
         * （明天、后天并入「本周」，下周并入「更晚」），所以首页不会漏掉任何一天。
         */
        val defaultEnabled: Set<HomeBucket> = setOf(TODAY, THIS_WEEK, LATER)

        fun byId(id: String): HomeBucket? = entries.firstOrNull { it.id == id }
    }
}

/**
 * Pure grouping rules for the home list. No Compose, no Android — unit-testable.
 *
 * Two layers:
 * 1. [HomeBoard] — 逾期 / 有日期 / 无日期, ordered & toggled by the user.
 * 2. [HomeBucket] — 今日 / 明天 / 后天 / 本周 / 下周 / 更晚, only inside 有日期.
 */
object HomeGrouping {

    /**
     * Which board a todo belongs to. [due] null = 无日期 (never overdue, never dated).
     *
     * [end] = 时间区间的结束点（#1，`endAtMillis`）：非空时逾期判定看结束点而不是开始点 ——
     * 「下午 3 点到 5 点」在 16:00 是**进行中**，不该进逾期板块，也不该标红。
     */
    fun boardOf(
```

```
    due: Long?,
    today: LocalDate = AppTime.today(),
    nowMillis: Long = AppTime.nowMillis(),
    hasExactTime: Boolean = true,
    end: Long? = null
): HomeBoard = when {
    due == null -> HomeBoard.UNDATED
    // Same single source of truth the red marking uses.
    Format.isOverdue(end ?: due, today, nowMillis, hasExactTime) -> HomeBoard.OVERDUE
    else -> HomeBoard.DATED
}

/**
 * The *natural* bucket of a dated, not-yet-overdue todo — ignoring which buckets the user
 * enabled. Today/tomorrow/day-after win over 本周 on purpose: they sit inside the current
 * natural week, so the week branch must never swallow them.
 */
fun bucketOf(due: Long, today: LocalDate = AppTime.today()): HomeBucket {
    val d = Format.dueAtMillis(due)
    val thisMonday = today.with(DayOfWeek.MONDAY)
    val thisSunday = today.with(DayOfWeek.SUNDAY)
    val nextMonday = thisMonday.plusWeeks(1)
    val nextSunday = thisSunday.plusWeeks(1)
    return when {
        d == today -> HomeBucket.TODAY
        d == today.plusDays(1) -> HomeBucket.TOMORROW
        d == today.plusDays(2) -> HomeBucket.DAY_AFTER
        !d.isBefore(thisMonday) && !d.isAfter(thisSunday) -> HomeBucket.THIS_WEEK
        !d.isBefore(nextMonday) && !d.isAfter(nextSunday) -> HomeBucket.NEXT_WEEK
        else -> HomeBucket.LATER
    }
}

/**
 * 时间段的分桶（v0.8.4 起）：
 * - **进行中**（today ∈ [start, end]）→ [HomeBucket.TODAY]：符合「今天要推进它」的直觉；
 * - **未开始** → 按开始日走 [bucketOf]；
 * - 已逾期由 [boardOf] 先分进逾期板块，这里不会被调用。
 * 单日（start == null）退回按结束日分桶，行为与旧版完全一致。
 */
fun bucketOfRange(
    start: Long?,
    end: Long?,
    today: LocalDate = AppTime.today()
): HomeBucket {
    if (start == null || end == null) {
        // 只有一端时按那一端归档：只有开始日的脏数据也必须有归属，不能返回"无桶"。
        val only = end ?: start ?: return HomeBucket.LATER
        return bucketOf(only, today)
    }
```

```
        val s = Format.dueAtMillis(start)
        val e = Format.dueAtMillis(end)
        if (!today.isBefore(s) && !today.isAfter(e)) return HomeBucket.TODAY
        return bucketOf(start, today)
    }

    /**
     * Map a todo's natural bucket onto the bucket it is rendered under.
     *
     * "Merge into the next enabled bucket" is what makes a partial selection like
     * `今日 + 本周 + 更晚` still cover every todo: tomorrow and the day after fall into 本周.
     *
     * @return null when nothing is selected — caller renders the board flat, without headers.
     */
    fun resolveBucket(original: HomeBucket, enabled: Set<HomeBucket>): HomeBucket? {
        if (enabled.isEmpty()) return null
        for (b in HomeBucket.all) {
            if (b.ordinal >= original.ordinal && b in enabled) return b
        }
        // Nothing later is enabled: the last enabled bucket catches everything beyond it.
        return HomeBucket.all.lastOrNull { it in enabled }
    }

    /**
     * Boards to render, in the user's order.
     *
     * Falling back to [HomeBoard.defaultOrder] when everything is hidden is deliberate: an
     * empty home screen with no way back would be a trap. The settings UI also refuses to
     * switch off the last visible board, so this is a second line of defence.
     */
    fun visibleBoards(
        order: List<HomeBoard> = HomeBoard.defaultOrder,
        hidden: Set<HomeBoard> = emptySet()
    ): List<HomeBoard> {
        val kept = order.filter { it !in hidden }
        return kept.ifEmpty { HomeBoard.defaultOrder }
    }
}
```

app/src/main/kotlin/com/lirui/oneline/util/Quotes.kt

```
package com.lirui.oneline.util

import android.content.Context

/**
 * 测试通知「详情」里那句文案。
 *
 * 为什么是随机一条：这条通知的作用是**让你在通知栏里看到真实排版**，
 * 每次点「发送测试通知」都换一句，才能顺带看出不同长度会不会被截断。
 *
 * 收录标准两条：
```

```
 * 1. **够短** —— 一行 / 最多两行能读完。长句一律不进这个池子：通知的 BigText 展开区在
 *    国产 ROM 上经常不给你点开的机会（小米/OPPO 折叠状态只显示 contentText），放长等于白放。
 * 2. **离开上下文能读懂** —— 所以「阳莽憾陨谁无败？卷土重来再称王」只收了后半句。
 *
 * v0.5 起这个池子是**可编辑的**（设置 → 提醒文案）：先读 SharedPreferences，
 * 没自定义过才回落到 [defaults]。所以任何取文案的地方都必须带 Context，
 * 不要再写死访问 [defaults] —— 那样用户改了半天不生效。
 *
 * 完整分组与长句见仓库根目录 `QUOTES.md`。
 */
object Quotes {

    /** 没有可用文案时的兜底正文。空池子不该让通知正文变成空白。 */
    const val FALLBACK = "该办了。"

    /** 内置默认池（通知详情用的短句）。用户在设置里改过就不再读它。 */
    val defaults: List<String> = listOf(
        "岂不闻天无绝人之路？只要我想走，路就在脚下。",
        "不过些许风霜罢了。",
        "男儿不展凌云志，空负天生八尺躯。",
        "今日暂且展翼去，明朝登仙笞凤凰。",
        "卷土重来再称王。",
        "千磨万击心铸铁，殚精竭虑铸一剑。",
        "早岁已知世事艰，仍许飞鸿荡云间。",
        "落魄谷中寒风吹，春秋蝉鸣少年归。",
        // 「坚持」这一组刻意分成三条：合成一段太长，通知里读不完。
        "我曾经呐喊过，渐渐地，我不发出声音……",
        "而如今！我只剩下面无表情，我的目光如磐石般坚硬，我的心中剩下坚持。",
        "这就是我，一个小人物的坚持。",
        "坚持蛊没有凡蛊，因为坚持本就不凡。",
        "希望蛊没有仙蛊，因为凡事都有希望。",
        "面对困境，就把心交给希望。",
        "有心的人，态度就是一张面具；无心的人，戴不上态度面具。",
        "自己才是世间最可靠的。",
        "最大的失败就是失去自我。",
        "人啊，即便弱小如我，也有生存的方法。",
        "规矩要在黑暗中摸索。"
    )

    /** 当前生效的文案池：用户自定义优先，否则 [defaults]。 */
    fun pool(context: Context): List<String> =
        SettingsRepo.customQuotes(context) ?: defaults

    /** 随机取一条（测试通知用）。 */
    fun random(context: Context): String =
        pool(context).ifEmpty { listOf(FALLBACK) }.random()

    /**
     * 按待办 id 取一条 —— **同一条待办永远是同一句**。
     *
```

```
     * 提醒通知的「完成 / 撤回」是同一个 notification id 的原地重发，
     * 用随机的话每点一次按钮详情就换一条文案，看起来像出了 bug。
     * 这里用 id 取模，天然稳定，也不用把文案塞进 PendingIntent 的 extra。
     *
     * 用户中途改了文案池，取模的基数会变 → 同一条待办的句子会换一次。
     * 这个取舍是接受的：改了文案还坚持用旧的，才是更奇怪的行为。
     */
    fun forTodo(context: Context, todoId: Long): String {
        val quotes = pool(context)
        if (quotes.isEmpty()) return FALLBACK
        val size = quotes.size
        val index = ((todoId % size) + size) % size
        return quotes[index.toInt()]
    }
}
```

app/src/main/kotlin/com/lirui/oneline/util/SettingsRepo.kt

```
package com.lirui.oneline.util

import android.content.Context

/**
 * Persists user settings (theme, reminder switches) via plain
 * [android.content.SharedPreferences] so the library set stays small and stable.
 *
 * Theme modes:
 *  - FOLLOW_SYSTEM : use the OS-wide setting (default)
 *  - LIGHT         : force light
 *  - DARK          : force dark
 */
object SettingsRepo {

    private const val PREF_NAME = "oneline_settings"
    private const val KEY_THEME = "theme_mode"
    private const val KEY_REMINDERS_ENABLED = "reminders_enabled"
    private const val KEY_REMIND_DUE = "remind_due"
    private const val KEY_REMIND_ADVANCE = "remind_advance"
    private const val KEY_OVERDUE_NAG = "overdue_nag"
    private const val KEY_OVERDUE_INTERVAL_HOURS = "overdue_interval_hours"
    // "overdue_pin" (逾期置顶) is retired as of v2.1 — board order now decides placement.
    // The stale key may still sit in existing installs; it is simply never read.
    private const val KEY_DEFAULT_TIME = "default_time"
    private const val KEY_AUTO_REPEAT = "auto_repeat"
    private const val KEY_PALETTE = "palette_id"
    private const val KEY_DARK_CARD = "dark_card_style"
    private const val KEY_IMMERSIVE_TOPBAR = "immersive_topbar"
    // 列表手势开关（设置 → 操作）。三个都是"用户可以关掉"的动作，
    // 关掉后对应手势完全不参与事件竞争（不是"接住了再判断"）。
    private const val KEY_GESTURE_SWIPE_ARCHIVE = "gesture_swipe_archive"
    private const val KEY_GESTURE_LONG_PRESS_EDIT = "gesture_long_press_edit"
    private const val KEY_GESTURE_DOUBLE_TAP_EDIT = "gesture_double_tap_edit"
```

```
    private const val KEY_HOME_BOARD_ORDER = "home_board_order"
    private const val KEY_HOME_BOARD_HIDDEN = "home_board_hidden"
    private const val KEY_HOME_BUCKETS = "home_buckets"
    private const val KEY_CUSTOM_QUOTES = "custom_quotes"
    private const val KEY_KEEP_ALIVE = "keep_alive"
    private const val KEY_KEEPALIVE_NOTIF_TITLE = "keepalive_notif_title"
    private const val KEY_KEEPALIVE_NOTIF_TEXT = "keepalive_notif_text"
    private const val KEY_WIDGET_SHOW_DATE = "widget_show_date"
    private const val KEY_WIDGET_DATE_BELOW = "widget_date_below"
    private const val KEY_WIDGET_EVENT_PRIORITY = "widget_event_priority"
    private const val KEY_WIDGET_DATE_ONLY = "widget_date_only"
    private const val KEY_WIDGET_HIDE_HEADER = "widget_hide_header"
    // 隐私政策：存的是「已同意的政策版本号」，0/缺失 = 从未同意。
    private const val KEY_PRIVACY_ACCEPTED_VERSION = "privacy_accepted_version"
    // 退役键（保留、永不读取）：widget_compact_show_date / widget_standard_show_date /
    // widget_show_weekday —— 见下方「桌面小组件」段落的说明。

    const val FOLLOW_SYSTEM = 0
    const val LIGHT = 1
    const val DARK = 2

    // ---------- theme ----------
    fun getThemeMode(context: Context): Int =
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE).getInt(KEY_THEME,
FOLLOW_SYSTEM)

    fun setThemeMode(context: Context, mode: Int) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putInt(KEY_THEME, mode).apply()
    }

    // ---------- colour palette ----------

    /** Selected palette id (see ui.theme.Palettes). Default 0 = 雅紫. */
    fun paletteId(context: Context): Int =
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE).getInt(KEY_PALETTE, 0)

    fun setPaletteId(context: Context, id: Int) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putInt(KEY_PALETTE, id).apply()
    }

    /**
     * 深色模式下的卡片底色档位（见 `ui.theme.DarkCardStyle`）。
     * 默认 **2 = 不染色**（纯灰底，靠圆角区分层次）。
     *
     * ★ id 不是按明度排的：NEUTRAL=2 / SOFT=1 / STANDARD=0 / RICH=3。
     *   看到"2"别以为是深彩 —— 深彩是 3。这里填错就是整页底色变样。
     */
    fun darkCardStyleId(context: Context): Int =
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE).getInt(KEY_DARK_CARD, 2)
```

```
fun setDarkCardStyleId(context: Context, id: Int) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putInt(KEY_DARK_CARD, id).apply()
}

// ---------- reminders ----------

/** Master switch for all reminders (default ON). */
fun remindersEnabled(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_REMINDERS_ENABLED, true)

/** Notify exactly at the due time (到点提醒). Default ON. */
fun remindAtDue(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_REMIND_DUE, true)

/** Notify in advance (graded D-7/D-3/D-1 + exact -1h). Default ON. */
fun remindInAdvance(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_REMIND_ADVANCE, true)

fun setRemindersEnabled(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_REMINDERS_ENABLED, enabled).apply()
}

fun setRemindAtDue(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_REMIND_DUE, enabled).apply()
}

fun setRemindInAdvance(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_REMIND_ADVANCE, enabled).apply()
}

// ---------- overdue nag (逾期反复叮嘱) ----------

/** Whether overdue, unfinished todos should be nagged repeatedly. Default ON. */
fun overdueNagEnabled(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_OVERDUE_NAG, true)

/** Nag interval in hours. Default 24h (daily). */
fun overdueIntervalHours(context: Context): Int =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getInt(KEY_OVERDUE_INTERVAL_HOURS, 24)
```

```
    fun setOverdueNagEnabled(context: Context, enabled: Boolean) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putBoolean(KEY_OVERDUE_NAG, enabled).apply()
    }

    fun setOverdueIntervalHours(context: Context, hours: Int) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putInt(KEY_OVERDUE_INTERVAL_HOURS, hours).apply()
    }

    // ---------- default reminder time (无日期/只写日期默认时刻) ----------

    /** Default time-of-day (minute-of-day) for a date-only todo. Default 09:00. */
    fun defaultMinuteOfDay(context: Context): Int =
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .getInt(KEY_DEFAULT_TIME, 9 * 60)

    fun setDefaultMinuteOfDay(context: Context, minuteOfDay: Int) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putInt(KEY_DEFAULT_TIME, minuteOfDay).apply()
    }

    // ---------- repeating todo rollover (循环重复待办) ----------

    /** Whether a repeating todo rolls to its next occurrence instead of archiving. Default ON.
*/
    fun autoRepeatEnabled(context: Context): Boolean =
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .getBoolean(KEY_AUTO_REPEAT, true)

    fun setAutoRepeatEnabled(context: Context, enabled: Boolean) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putBoolean(KEY_AUTO_REPEAT, enabled).apply()
    }

    // ---------- 沉浸式顶栏 (immersive top bar) ----------
    //
    // 开 = 列表上滑时顶栏跟着收起，内容能滚到状态栏底下（浮层顶栏）；
    // 关 = 顶栏固定不动，永远是「顶部标题栏」的老样子。
    //
    // ★ 默认**关**（zuige 2026-10-23 定：首装默认不开启沉浸式）。
    //   这个键在 v0.6 曾被退役过（浮层收起一度成为固定行为），现在重新接回来 ——
    //   老用户 prefs 里残留的 immersive_topbar 值会被重新读到，这正是我们要的
    //   （他们当初关掉过，就该保持关掉）。
    fun immersiveTopBar(context: Context): Boolean =
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .getBoolean(KEY_IMMERSIVE_TOPBAR, false)

    fun setImmersiveTopBar(context: Context, enabled: Boolean) {
        context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
            .edit().putBoolean(KEY_IMMERSIVE_TOPBAR, enabled).apply()
```

```
}

// ---------- 列表手势（v0.9.1）----------
//
// 三个动作互相独立，默认**全开**（老用户的行为保持不变；双击是新增的，
// 单击本来就没有动作，所以不存在"抢点击"的问题）。
// 关掉的手势在 TodoRow 里连 pointerInput 都不挂 —— 不占事件分发的竞争。

/** 右滑归档。默认开。 */
fun gestureSwipeArchive(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_GESTURE_SWIPE_ARCHIVE, true)

fun setGestureSwipeArchive(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_GESTURE_SWIPE_ARCHIVE, enabled).apply()
}

/** 长按编辑。默认开。 */
fun gestureLongPressEdit(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_GESTURE_LONG_PRESS_EDIT, true)

fun setGestureLongPressEdit(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_GESTURE_LONG_PRESS_EDIT, enabled).apply()
}

/** 双击编辑。默认开。 */
fun gestureDoubleTapEdit(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_GESTURE_DOUBLE_TAP_EDIT, true)

fun setGestureDoubleTapEdit(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_GESTURE_DOUBLE_TAP_EDIT, enabled).apply()
}

// ---------- 后台常驻 (keep-alive FGS) ----------
//
// 只记录"用户想要什么"，不记录"服务现在在不在跑"。
// 真实状态以 `KeepAlive.isRunning`（进程内标志）+ 系统服务列表为准 ——
// 把两者混成一个键，就会出现"设置里是开的、服务其实没起来"的假象。

/**
 * 用户是否打开了「后台常驻」。默认**开**（zuige 2026-10-23 定：首装即常驻）。
 *
 * ★ 这里只记录"用户想要什么"，不记录服务在不在跑（见上面那段说明）。
 *   默认开意味着首装会挂一条常驻通知 —— 这是有意为之：提醒可靠性优先，
 *   用户可以在设置里关掉。
```

```
 */
fun keepAliveEnabled(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_KEEP_ALIVE, true)

fun setKeepAliveEnabled(context: Context, enabled: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_KEEP_ALIVE, enabled).apply()
}

// ---------- 常驻通知文案（后台常驻那条通知的标题/正文） ----------
//
// 默认值与 KeepAliveService 里原先硬编码的字符串一致 —— 改这里不会让
// "升级后通知文案突然变了"。空串是合法值：用户清空标题时用应用名兜底
//（通知标题为空会被系统折叠成包名，反而难看）。

// 默认值 = zuige 当前在用的那套（2026-10-23 定为首装默认）。
// 正文默认空串是**有意**的：只留标题一行，不重复说"点击查看待办"。
const val DEFAULT_KEEPALIVE_TITLE = "常驻服务进行中"
const val DEFAULT_KEEPALIVE_TEXT = ""

fun keepAliveNotifStyle(context: Context): KeepAliveNotifStyle {
    val p = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
    return KeepAliveNotifStyle(
        title = p.getString(KEY_KEEPALIVE_NOTIF_TITLE, null) ?: DEFAULT_KEEPALIVE_TITLE,
        body = p.getString(KEY_KEEPALIVE_NOTIF_TEXT, null) ?: DEFAULT_KEEPALIVE_TEXT
    )
}

fun saveKeepAliveNotif(context: Context, title: String, body: String) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit()
        .putString(KEY_KEEPALIVE_NOTIF_TITLE, title)
        .putString(KEY_KEEPALIVE_NOTIF_TEXT, body)
        .apply()
}

// ---------- 桌面小组件 ----------
//
// ★ 2026-10-23 重构：**不再按组件尺寸分档**。
//   原因：vivo 桌面（com.bbk.launcher2）报的 OPTION_APPWIDGET_MIN/MAX_HEIGHT 恒为 152px，
//   不管组件拉到 2×2 还是 2×4 —— 按高度判档位会把 2×4 也判成 2×2，
//   设置里"我明明是 2×4 却说我是 2×2"就是这么来的。
//   现在日期怎么显示完全由下面三个开关决定，与尺寸无关：
//     widget_show_date    是否显示日期（总开关）
//     widget_date_below   true = 日期在标题下方（事件优先）；false = 日期在右侧（日期优先）
//     widget_date_only    true = 只显示「10 月 23 日」，不带「14 点」
//   旧键 widget_compact_show_date / widget_standard_show_date / widget_show_weekday
//   一并退役（脏键保留、永不读取）。
```

```
/** 是否显示日期。默认开。 */
fun widgetShowDate(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_WIDGET_SHOW_DATE, true)

fun setWidgetShowDate(context: Context, value: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_WIDGET_SHOW_DATE, value).apply()
}

/**
 * 日期放哪：true = 标题下方（事件占满整行宽度，**事件优先**）；
 * false = 标题右侧（日期完整显示、事件太长才省略，**日期优先**）。
 *
 * 默认 **false（右侧）** —— zuige 2026-10-23 定：首装默认日期靠右。
 */
fun widgetDateBelow(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_WIDGET_DATE_BELOW, false)

fun setWidgetDateBelow(context: Context, value: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_WIDGET_DATE_BELOW, value).apply()
}

/**
 * 日期在**右侧**时，谁优先占宽度（v0.9.1 补丁）。
 *
 * true = **事件优先**：标题先拿到完整宽度，日期只吃剩下的、超出就省略号；
 * false = 日期优先：日期完整，标题被压缩（这就是"通知测…"那种观感）。
 *
 * ★ 只在 `widgetDateBelow == false` 时生效 —— 日期放到标题下方之后，标题本来就独占
 *   整行宽度，没有"谁让谁"的问题了（zuige 明确：日期在下面不用管这个开关）。
 *
 * 默认 **true（事件优先）** —— zuige 2026-10-23 定：默认事件覆盖日期。
 */
fun widgetEventPriority(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_WIDGET_EVENT_PRIORITY, true)

fun setWidgetEventPriority(context: Context, value: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_WIDGET_EVENT_PRIORITY, value).apply()
}

/** 只显示日期、不带具体时间（「10 月 23 日」而不是「10 月 23 日 14 点」）。默认开。 */
fun widgetDateOnly(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_WIDGET_DATE_ONLY, true)
```

```
fun setWidgetDateOnly(context: Context, value: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_WIDGET_DATE_ONLY, value).apply()
}

/** 是否隐藏组件头部（「待办」标题 + 条数）。默认**隐藏**（zuige 当前配置）。 */
fun widgetHideHeader(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getBoolean(KEY_WIDGET_HIDE_HEADER, true)

fun setWidgetHideHeader(context: Context, value: Boolean) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putBoolean(KEY_WIDGET_HIDE_HEADER, value).apply()
}

// ---------- 提醒文案池 (custom quotes) ----------
//
// 存成单个字符串（`\n` 分隔）而不是 StringSet：
// 1. StringSet 丢顺序，用户排在第一位的那句会随机跑位；
// 2. 一条文案本身不可能含换行（通知正文是单行），所以换行是安全的分隔符。
//
// 语义：**键不存在 = 用内置默认池**；键存在但为空串 = 用户把文案删光了。
// 后者不写键（见 setCustomQuotes），所以实际只有"键不存在"和"非空列表"两种状态。

/**
 * @return null = 没自定义过，调用方应使用 [Quotes.defaults]
 */
fun customQuotes(context: Context): List<String>? {
    val prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
    if (!prefs.contains(KEY_CUSTOM_QUOTES)) return null
    val raw = prefs.getString(KEY_CUSTOM_QUOTES, "").orEmpty()
    return raw.split('\n').map { it.trim() }.filter { it.isNotEmpty() }
}

/**
 * 保存自定义文案。
 *
 * 清空 = 恢复默认（删掉键），而不是"一句都没有"—— 空池子会让通知正文变成空白，
 * 那是用户不会想要的副作用。
 */
fun setCustomQuotes(context: Context, quotes: List<String>) {
    val cleaned = quotes.map { it.trim() }.filter { it.isNotEmpty() }.distinct()
    val edit = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE).edit()
    if (cleaned.isEmpty()) {
        edit.remove(KEY_CUSTOM_QUOTES)
    } else {
        edit.putString(KEY_CUSTOM_QUOTES, cleaned.joinToString("\n"))
    }
    edit.apply()
}
```

```
// ---------- home list grouping (首页分组) ----------
//
// Boards are persisted as a comma-separated id list so the ORDER survives; a StringSet
// would lose it. Buckets store an explicit empty string when the user unticks everything,
// which must stay distinguishable from "never configured" (that one means "all on").

/** Board order, always complete: unknown/missing ids are appended in the default order. */
fun homeBoardOrder(context: Context): List<HomeBoard> {
    val raw = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getString(KEY_HOME_BOARD_ORDER, null) ?: return HomeBoard.defaultOrder
    val parsed = raw.split(",").mapNotNull { HomeBoard.byId(it.trim()) }.distinct()
    return parsed + HomeBoard.defaultOrder.filter { it !in parsed }
}

fun setHomeBoardOrder(context: Context, order: List<HomeBoard>) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putString(KEY_HOME_BOARD_ORDER, order.joinToString(",") { it.id }).apply()
}

/** Boards switched off on the home page. */
fun hiddenBoards(context: Context): Set<HomeBoard> {
    val raw = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getString(KEY_HOME_BOARD_HIDDEN, null) ?: return emptySet()
    if (raw.isBlank()) return emptySet()
    return raw.split(",").mapNotNull { HomeBoard.byId(it.trim()) }.toSet()
}

fun setHiddenBoards(context: Context, hidden: Set<HomeBoard>) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putString(KEY_HOME_BOARD_HIDDEN, hidden.joinToString(",") { it.id }).apply()
}

/** Enabled buckets inside 有日期. Empty set = don't sub-group, render the board flat. */
fun homeBuckets(context: Context): Set<HomeBucket> {
    val prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
    if (!prefs.contains(KEY_HOME_BUCKETS)) return HomeBucket.defaultEnabled
    val raw = prefs.getString(KEY_HOME_BUCKETS, "").orEmpty()
    if (raw.isBlank()) return emptySet()
    return raw.split(",").mapNotNull { HomeBucket.byId(it.trim()) }.toSet()
}

fun setHomeBuckets(context: Context, buckets: Set<HomeBucket>) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putString(KEY_HOME_BUCKETS, buckets.joinToString(",") { it.id }).apply()
}

// ---------- notification title (通知标题) ----------
//
// v0.4 起版式从「四段拼接」改成「一个模板 + 一个完成后缀」:
```

```
    //
    //    旧：  事件 + 空格 + lead + dueWord + tail          （三段各自一个设置项，易错位）
    //    新：  事件 + 空格 + template.replace({提醒}, 状态语)
    //
    // `{提醒}` 是**占位符**，不是固定文字。原因：通知标题里的状态语随提醒类型变化
    // （提前提醒 / 1 小时后到期 / 该办了 / 还没完成，别忘了哦，见 ReminderReceiver），
    // 若模板写死「该办了」，提前提醒也会说"该办了"，语义就错了。
    // 用户把 `{提醒}` 删掉是合法的 —— 那时四种提醒共用同一句固定标题，由用户自己决定。
    //
    // 旧键（notif_lead_emoji / notif_tail_emoji / notif_due_word）保留**只读**：
    // `dueWord` 仍是 DUE 状态语的取值（迁移不丢用户设置），lead/tail 只在首次迁移时
    // 拼进模板，之后不再读取。别把它们捡回来当第二套写入路径 —— 那正是"框里还有、
    // 通知里没有"这类不一致的温床。

    private const val KEY_NOTIF_TITLE_TEMPLATE = "notif_title_template"
    private const val KEY_NOTIF_LEAD = "notif_lead_emoji"
    private const val KEY_NOTIF_TAIL = "notif_tail_emoji"
    private const val KEY_NOTIF_DUE_WORD = "notif_due_word"
    private const val KEY_NOTIF_DONE = "notif_done_emoji"

    /** 模板里的状态语占位符。改这个常量等于破坏已存数据，别动。 */
    const val NOTIF_REMINDER_TOKEN = "{提醒}"

    // 默认值 = zuige 当前在用的那套（2026-10-23 定为首装默认）：
    //    模板「 🚀🚀{提醒}🚀🚀」、完成后缀「 🎉🎉」、DUE 状态语「该办了」。
    // 注意 LEAD/TAIL 只影响**老用户迁移**（模板键不存在时才拼），改它们不会动已有用户。
    const val DEFAULT_NOTIF_DUE_WORD = "该办了"
    const val DEFAULT_NOTIF_LEAD = " 🚀🚀"
    const val DEFAULT_NOTIF_TAIL = "🚀🚀"
    const val DEFAULT_NOTIF_DONE = " 🎉🎉"

    /** 模板 + 完成后缀的默认值（与迁移结果一致）。 */
    const val DEFAULT_NOTIF_TEMPLATE = DEFAULT_NOTIF_LEAD + NOTIF_REMINDER_TOKEN +
DEFAULT_NOTIF_TAIL

    /**
     * 读取「标题模板」，**顺带完成旧数据迁移**。
     *
     * 迁移只在第一次读时发生（写回 `notif_title_template` 后就不再计算），
     * 所以用户之后怎么改 lead/tail 都不影响模板 —— 单一数据源。
     */
    fun notifTitleTemplate(context: Context): String {
        val prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        val existing = prefs.getString(KEY_NOTIF_TITLE_TEMPLATE, null)
        if (existing != null) return existing

        // 旧数据：lead + 状态语 + tail 拼成模板。空串是合法值（用户清空过），不能回退默认。
        val lead = prefs.getString(KEY_NOTIF_LEAD, DEFAULT_NOTIF_LEAD).orEmpty()
        val tail = prefs.getString(KEY_NOTIF_TAIL, DEFAULT_NOTIF_TAIL).orEmpty()
        val template = lead + NOTIF_REMINDER_TOKEN + tail
        prefs.edit().putString(KEY_NOTIF_TITLE_TEMPLATE, template).apply()
```

```
    return template
}

/** 一次性保存「标题模板 + 完成后缀」—— 与设置页那一个保存按钮一一对应。 */
fun saveNotificationTitle(context: Context, template: String, done: String) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit()
        .putString(KEY_NOTIF_TITLE_TEMPLATE, template)
        .putString(KEY_NOTIF_DONE, done)
        .apply()
}

/** 一次读全，避免每发一条通知就查三遍 SharedPreferences。 */
fun notificationStyle(context: Context): NotificationStyle {
    val p = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
    return NotificationStyle(
        titleTemplate = notifTitleTemplate(context),
        done = p.getString(KEY_NOTIF_DONE, DEFAULT_NOTIF_DONE).orEmpty(),
        dueWord = p.getString(KEY_NOTIF_DUE_WORD, DEFAULT_NOTIF_DUE_WORD).orEmpty()
    )
}

/**
 * DUE 状态语（"该办了"）—— 不再是独立设置项，迁移旧值时写入，
 * 目前只在测试通知和设置页默认值里用到。
 */
fun setNotifDueWord(context: Context, value: String) =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putString(KEY_NOTIF_DUE_WORD, value).apply()

// ---------- 隐私政策同意（v1.0.0 起） ----------
//
// 记的是**已同意的政策版本号**，不是一个 boolean。理由：政策改版后要能重新征得同意，
// 而 boolean 只能表达"同意过"，没法区分"同意的是哪一版"。
//
// 首次启动流程见 MainActivity：未同意 → 只渲染 ConsentDialog，不申请任何权限、
// 不排闹钟、不建数据库以外的任何副作用。

/** 当前隐私政策版本。政策内容有实质变更时 +1，老用户会被重新要求同意。 */
const val PRIVACY_POLICY_VERSION = 1

/** 用户是否已同意**当前版本**的隐私政策。 */
fun privacyAccepted(context: Context): Boolean =
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .getInt(KEY_PRIVACY_ACCEPTED_VERSION, 0) >= PRIVACY_POLICY_VERSION

fun setPrivacyAccepted(context: Context) {
    context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        .edit().putInt(KEY_PRIVACY_ACCEPTED_VERSION, PRIVACY_POLICY_VERSION).apply()
}
```

```
}

/**
 * 通知标题的样式。
 *
 * - [titleTemplate]：事件名之后的部分，`{提醒}` 会被替换成当前提醒类型的状态语；
 * - [done]：勾选完成后整条标题换成 `事件 + 空格 + done`；
 * - [dueWord]：DUE 那一类的状态语（"该办了"），模板里通过 `{提醒}` 展开。
 *
 * 任何一个字段为空串都表示「这一段不要」，不是「回退默认」。
 */
data class NotificationStyle(
    val titleTemplate: String,
    val done: String,
    val dueWord: String
)

/** 常驻通知（后台常驻服务）的标题与正文。 */
data class KeepAliveNotifStyle(
    val title: String,
    val body: String
)
```

app/src/main/kotlin/com/lirui/oneline/widget/TodoWidgetActionReceiver.kt

```
package com.lirui.oneline.widget

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import com.lirui.oneline.data.TodoRepository
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

/**
 * 小组件「圆圈完成」的独立接收器（exported=false）。
 *
 * ★ 为什么从 [TodoWidgetProvider] 拆出来：v0.8.1 起主 provider 改为 exported=true
 *   （部分国产 Launcher bind 组件时不是特权身份，exported=false 会被拒 → 组件
 *   进不了系统列表，澎湃 3/vivo 实测）。但 COMPLETE 动作**不能**跟着 exported=true ——
 *   那等于任何应用都能广播一条 todoId 把待办"完成"掉。
 *
 *   拆开后：圆圈的 PendingIntent 用**显式 ComponentName** 指到这里。non-exported 的
 *   receiver 可以收到我们自己（同 UID）发出的广播，系统投递没问题。
 *
 * ★ 归档为什么分两帧（见 [WidgetUpdater.markCompleting]）：
 *   RemoteViews 没有动画 API，只能靠两次 updateAppWidget 拼出"勾亮起 → 行消失"。
 *   第 1 帧只改外观（不写库），第 2 帧才 archive + 刷新 —— 所以第 1 帧的 refresh
 *   必须在 archive **之前**，否则查出来这条已经没了，两帧会变成"什么都没发生"。
 */
class TodoWidgetActionReceiver : BroadcastReceiver() {
```

```
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action != TodoWidgetProvider.ACTION_COMPLETE) return
        val id = intent.getLongExtra(TodoWidgetProvider.EXTRA_TODO_ID, -1L)
        if (id <= 0L) return
        // goAsync 把广播生命周期拉到协程结束（含中间的 300ms delay），
        // 否则 onReceive 一返回系统就可能回收进程，第 2 帧永远不执行。
        val pendingResult = goAsync()
        val app = context.applicationContext
        WidgetUpdater.ioScope.launch {
            try {
                archiveWithTick(app, id)
            } finally {
                // 失败也要清掉瞬时状态，否则这条会一直画着勾。
                WidgetUpdater.clearCompleting()
                pendingResult.finish()
            }
        }
    }
}

/**
 * 「勾亮起 → 行消失」的两帧归档。**必须在 IO 线程调用**（内部同步查库）。
 *
 * 第 1 帧只改外观：`markCompleting` + refresh，这条被画成已勾选；
 * 第 2 帧才写库：archive 之后清掉标记再 refresh，这一行从列表里消失、后面的补位。
 *
 * ★ 顺序不能反：先 archive 再 refresh，第 1 帧查库时这条已经没了，两帧就退化成
 *   "什么都没发生"（用户看到的还是"没有动效"）。
 */
internal suspend fun archiveWithTick(context: Context, id: Long) {
    WidgetUpdater.markCompleting(id)
    WidgetUpdater.refresh(context)
    delay(WidgetUpdater.TICK_MILLIS)
    TodoWidgetRepo.get(context).archive(id)
    WidgetUpdater.clearCompleting()
    WidgetUpdater.refresh(context)
}
```

app/src/main/kotlin/com/lirui/oneline/widget/TodoWidgetProvider.kt

```
package com.lirui.oneline.widget

import android.app.PendingIntent
import android.appwidget.AppWidgetManager
import android.appwidget.AppWidgetProvider
import android.content.ComponentName
import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.SystemClock
import android.util.Log
import android.view.View
```

```
import android.widget.RemoteViews
import androidx.compose.ui.graphics.toArgb
import com.lirui.oneline.MainActivity
import com.lirui.oneline.R
import com.lirui.oneline.data.AppDatabase
import com.lirui.oneline.data.model.Todo
import com.lirui.oneline.ui.theme.Palettes
import com.lirui.oneline.util.AppTime
import com.lirui.oneline.util.Format
import com.lirui.oneline.util.SettingsRepo
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.launch
import kotlinx.coroutines.runBlocking

/**
 * 桌面小组件：一个 provider，纵横都可拉伸，**恒显示最近 4 条**。
 *
 * ★ 2026-10-23 起**不再按组件尺寸切布局**：vivo 桌面（com.bbk.launcher2）报的
 *   OPTION_APPWIDGET_MIN/MAX_HEIGHT 恒为 152px，2×2 和 2×4 一模一样 —— 按高度判档位
 *   必然判错（用户把组件拉到 2×4，设置里还说"当前是 2×2"）。
 *   现在日期怎么显示由设置里的三个开关决定，与尺寸无关：
 *     widget_show_date / widget_date_below / widget_date_only（见 SettingsRepo）
 *
 * ★ 架构（v0.8.1 重构）：**静态多行 RemoteViews**（removeAllViews + addView），
 *   不是 ListView + RemoteViewsService。
 *   原因：vivo 桌面实测从不 bind 数据服务——onDataSetChanged 一次都不跑，列表恒空
 *   且无任何报错。行数本来就是固定的（4 条），静态塞行完全覆盖需求，
 *   还消灭了 service/bind/factory 这一整层 ROM 兼容性问题。
 *   ★ 每次更新必须 `removeAllViews` 再 `addView`：桌面会复用视图树，addView 是追加，
 *     漏了清空就会堆出多余的历史行（表现为"日期关了还在""行数对不上"）。
 */
class TodoWidgetProvider : AppWidgetProvider() {

    override fun onUpdate(
        context: Context,
        manager: AppWidgetManager,
        appWidgetIds: IntArray
    ) {
        // 系统 30 分钟一次的兜底刷新（appwidget-info 的 updatePeriodMillis）。
        // ★ onUpdate 在主线程；updateOne 里要查 Room —— Room 禁止主线程访问，
        //   必须在 IO 上跑。goAsync 把广播生命周期拉到协程结束。
        val pendingResult = goAsync()
        WidgetUpdater.ioScope.launch {
            try {
                appWidgetIds.forEach { WidgetUpdater.updateOne(context, manager, it) }
            } finally {
                pendingResult.finish()
            }
```

```
        }
    }

    override fun onAppWidgetOptionsChanged(
        context: Context,
        manager: AppWidgetManager,
        appWidgetId: Int,
        newOptions: android.os.Bundle
    ) {
        // 用户拖大了组件：重新按高度选行布局（条数恒 4）。同样要在 IO 上查库。
        val pendingResult = goAsync()
        WidgetUpdater.ioScope.launch {
            try {
                WidgetUpdater.updateOne(context, manager, appWidgetId)
            } finally {
                pendingResult.finish()
            }
        }
    }

    companion object {
        /** 完成动作由 [TodoWidgetActionReceiver]（exported=false）接，见其 KDoc。 */
        const val ACTION_COMPLETE = "com.lirui.oneline.widget.action.COMPLETE"
        const val EXTRA_TODO_ID = "todo_id"
    }
}

/**
 * 组件刷新入口。App 内的写操作（增删改、归档、撤销）都调 [refreshAsync]；
 * Provider 自己（onUpdate / 完成动作）在 IO 协程里直接跑 [refresh]。
 */
object WidgetUpdater {

    private const val TAG = "TodoWidget"

    /** fire-and-forget 的 IO 作用域。写库后刷新组件不应拖累调用方（可能是主线程）。 */
    val ioScope = CoroutineScope(SupervisorJob() + Dispatchers.IO)

    fun refreshAsync(context: Context) {
        ioScope.launch { refresh(context) }
    }

    /** 同步刷新全部组件。**禁止在主线程调用**（内部同步查 Room）。 */
    fun refresh(context: Context) {
        val manager = AppWidgetManager.getInstance(context) ?: return
        val ids = manager.getAppWidgetIds(ComponentName(context,
TodoWidgetProvider::class.java))
        if (ids.isEmpty()) return
        ids.forEach { updateOne(context, manager, it) }
    }
```

```
/**
 * 组件最多显示几条待办。**固定 4 条**（zuige 2026-10-23 要求）：
 * 不随尺寸、也不随「隐藏头部」变化 —— 隐藏头部只省 26dp，不该变成"多显示一条"。
 */
private const val MAX_ROWS = 4

// ---------- 「两帧归档」的瞬时状态 ----------
//
// RemoteViews **没有任何动画 API**（没有 ViewPropertyAnimator、没有 ObjectAnimator，
// 系统只接受"整棵视图树的一次快照"）。所以组件里的归档反馈只能靠两次
// updateAppWidget 拼出来：
//   第 1 帧（t=0）     ：这条画成"已勾选"（圆圈里出现勾 + 标题变灰）→ 点击有即时反馈
//   第 2 帧（t=+300ms）：真正归档并刷新 → 这一行消失，下面的补位
// 中间这 300ms 就是"动效"：用户看到勾亮起，再看到行被收走。
//
// 状态放在进程内的 @Volatile 字段，而不是 SharedPreferences：一次归档的生命周期
// 只有 300ms，落盘既慢又没必要；万一进程在这中间被杀，[COMPLETING_TTL_MS] 的
// 时间窗保证下次读到的旧 id 会被忽略（不会把某条永远画成已勾选）。

/** 第 1 帧与第 2 帧之间的间隔。太短桌面来不及重绘，太长点击会"发木"。 */
const val TICK_MILLIS = 300L

/** 瞬时状态的有效期。超过它说明第 2 帧没跑成，当作没有。 */
private const val COMPLETING_TTL_MS = 2_000L

@Volatile
private var completingId: Long = -1L

@Volatile
private var completingAt: Long = 0L

/** 标记"这条正在播放归档"，下一次 [refresh] 会把它画成已勾选。 */
fun markCompleting(id: Long) {
    completingId = id
    completingAt = SystemClock.elapsedRealtime()
}

fun clearCompleting() {
    completingId = -1L
}

private fun isCompleting(id: Long): Boolean =
    id == completingId && SystemClock.elapsedRealtime() - completingAt < COMPLETING_TTL_MS

fun updateOne(context: Context, manager: AppWidgetManager, appWidgetId: Int) {
    try {
        // ★ 不再读 AppWidgetOptions 判尺寸：vivo 桌面报的 MIN/MAX_HEIGHT 恒为 152px，
        //   2×2 和 2×4 一模一样（实测），按高度分档必然判错。
        //   日期怎么显示完全由设置里的三个开关决定，与尺寸无关。
        val showDate = SettingsRepo.widgetShowDate(context)
```

```
val dateBelow = SettingsRepo.widgetDateBelow(context)
val eventPriority = SettingsRepo.widgetEventPriority(context)
val dateOnly = SettingsRepo.widgetDateOnly(context)
val hideHeader = SettingsRepo.widgetHideHeader(context)

val dao = AppDatabase.get(context).todoDao()
// ★ 先拿全量再截断：头部「N 条」统计的是**活跃待办总数**，不是"这一屏放得下几条"。
//   以前直接 take(MAX_ROWS) 后再取 size，于是有 7 条时头部写「4 条」——
//   用户以为数据丢了。
val all = runBlocking { dao.getActiveAll() }
val todos = all.take(MAX_ROWS)

val rv = RemoteViews(context.packageName, R.layout.widget_todo_list)
rv.setTextViewText(
    R.id.widget_count,
    context.getString(R.string.widget_header_count, all.size)
)
rv.setViewVisibility(
    R.id.widget_header,
    if (hideHeader) View.GONE else View.VISIBLE
)

if (todos.isEmpty()) {
    rv.removeAllViews(R.id.widget_rows)
    rv.setViewVisibility(R.id.widget_rows, View.GONE)
    rv.setViewVisibility(R.id.widget_empty, View.VISIBLE)
} else {
    rv.setViewVisibility(R.id.widget_rows, View.VISIBLE)
    rv.setViewVisibility(R.id.widget_empty, View.GONE)
    // ★★ 必须先清空容器再 addView。桌面（vivo 实测）会复用已 inflate 的视图树，
    //    addView 是**追加**而不是替换 —— 少了这句，历史更新的行会一直堆在下面：
    //    于是"日期已经关掉"的新行在前、还带着日期的旧行在后（被裁到可视区边缘），
    //    看起来就是「设置里关了，组件上还是显示时间」，行数也会多出来。
    //    类注释里一直写着 removeAllViews + addView，实现里却漏了这一句。
    rv.removeAllViews(R.id.widget_rows)
    todos.forEach { todo ->
        rv.addView(
            R.id.widget_rows,
            buildRow(
                context,
                todo,
                showDate,
                dateBelow,
                eventPriority,
                dateOnly,
                done = isCompleting(todo.id)
            )
        )
    }
}
```

```
        // 点头部（列表以外的空白也算）→ 打开 App。
        // ★ 打开 App 的 Intent 必须显式带上「复用已有实例」的 flag。
        //   小组件走的是 PendingIntent.getActivity，系统会补 FLAG_ACTIVITY_NEW_TASK，
        //   但那只决定"用哪个 task"，不决定"要不要新压一个 MainActivity"——
        //   standard 启动模式下每次点击都会再压一个实例，用户按返回会看到
        //   "还有一个主页面"，要点两次才退到桌面。
        val openApp = PendingIntent.getActivity(
            context,
            appWidgetId,
            Intent(context, MainActivity::class.java).apply {
                flags = Intent.FLAG_ACTIVITY_NEW_TASK or
                    Intent.FLAG_ACTIVITY_CLEAR_TOP or
                    Intent.FLAG_ACTIVITY_SINGLE_TOP
            },
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        rv.setOnClickPendingIntent(R.id.widget_header, openApp)
        rv.setOnClickPendingIntent(R.id.widget_empty, openApp)

        manager.updateAppWidget(appWidgetId, rv)
        Log.d(
            TAG,
            "updateOne id=$appWidgetId rows=${todos.size} showDate=$showDate " +
                "dateBelow=$dateBelow dateOnly=$dateOnly hideHeader=$hideHeader " +
                "done=${completingId}"
        )
    } catch (error: Throwable) {
        // 组件更新失败绝不能带崩广播（上次主线程查库就是这么炸出"空白组件"的）。
        Log.e(TAG, "updateOne id=$appWidgetId failed", error)
    }
}

/**
 * 构建一行。两个 PendingIntent 挂不同 view：
 * 整行 = 打开 App；圆圈 = 完成广播（独立 receiver，见 TodoWidgetActionReceiver）。
 *
 * 三张布局按设置选（**不再按组件尺寸**，vivo 报的尺寸不可信）：
 *  - [showDate]=false      → widget_item_todo_nodate（布局里根本没有 widget_due，
 *                            不靠 setViewVisibility —— vivo 复用行视图时会藏不住）
 *  - [dateBelow]=true      → widget_item_todo_below（日期在标题下方，标题独占整行）
 *  - [eventPriority]=true  → widget_item_todo_event（日期在右，**事件优先**：标题先占宽度）
 *  - 否则                   → widget_item_todo（日期在右，**日期优先**：日期完整、标题让位）
 *
 * [done]=true 是归档第 1 帧：圆圈换成"带勾的圈"、标题转灰，300ms 后整行消失。
 */
private fun buildRow(
    context: Context,
    todo: Todo,
    showDate: Boolean,
```

```
    dateBelow: Boolean,
    eventPriority: Boolean,
    dateOnly: Boolean,
    done: Boolean = false
): RemoteViews {
    val layout = when {
        !showDate -> R.layout.widget_item_todo_nodate
        dateBelow -> R.layout.widget_item_todo_below
        // 日期在右侧时才有"谁让谁"的问题：
        //   事件优先 → 标题先占宽度，日期吃剩下的（超出省略）
        //   日期优先 → 日期完整，标题提前省略（"通知测…"）
        eventPriority -> R.layout.widget_item_todo_event
        else -> R.layout.widget_item_todo
    }
    val rv = RemoteViews(context.packageName, layout)
    rv.setTextViewText(R.id.widget_title, todo.title)

    // 圆圈描边跟随 App 里选的配色（设置 → 外观 → 配色）。
    // shape 是空心圆，setColorFilter 会整体着色 → 描边变成当前主题主色。
    // 已勾选态（widget_circle_done 是单色 vector）同样靠这个 filter 上色。
    rv.setInt(
        R.id.widget_circle,
        "setColorFilter",
        Palettes.byId(SettingsRepo.paletteId(context)).primary.toArgb()
    )
    if (done) {
        rv.setImageViewResource(R.id.widget_circle, R.drawable.widget_circle_done)
        // 标题转灰 = "已经处理掉了"，和 App 里 progress 把标题淡化的观感对齐。
        rv.setTextColor(
            R.id.widget_title,
            context.getColor(R.color.widget_text_secondary)
        )
    }

    // 到期文案 + 逾期标红。无日期（dueAtMillis == null）时右侧留空。
    // 组件**不带星期**（zuige 2026-10-23：只要日期），故 showWeekday 恒 false。
    val due = todo.dueAtMillis
    if (due != null && showDate) {
        val (label, overdue) = dueLabel(
            due,
            todo.hasExactTime,
            showTime = !dateOnly,
            startAtMillis = todo.startAtMillis,
            endAtMillis = todo.endAtMillis
        )
        rv.setTextViewText(R.id.widget_due, label)
        rv.setTextColor(
            R.id.widget_due,
            context.getColor(
                if (overdue) R.color.widget_overdue else R.color.widget_text_secondary
```

```
                )
            )
        }

        // 整行 → 打开 App（带 id，先留着给将来的"跳到该待办"）。
        // requestCode 用 todo.id：不同行的 PendingIntent 不能互相复用，否则 extras 串。
        val open = Intent(context, MainActivity::class.java).apply {
            putExtra("todoId", todo.id)
            data = Uri.parse("todo://open/${todo.id}")
            // 同 widget_header：不让每次点击都新压一个 MainActivity（见上面 openApp 的注释）。
            // data 不同会让 Intent 视为不同，所以 flag 必须写在 Intent 自己身上。
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or
                Intent.FLAG_ACTIVITY_CLEAR_TOP or
                Intent.FLAG_ACTIVITY_SINGLE_TOP
        }
        rv.setOnClickPendingIntent(
            R.id.widget_row,
            PendingIntent.getActivity(
                context,
                todo.id.toInt(),
                open,
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
            )
        )

        // 行首圆圈 → 直接完成（显式 ComponentName 投递给 exported=false 的 receiver）。
        val complete = Intent(context, TodoWidgetActionReceiver::class.java).apply {
            action = TodoWidgetProvider.ACTION_COMPLETE
            putExtra(TodoWidgetProvider.EXTRA_TODO_ID, todo.id)
            data = Uri.parse("todo://complete/${todo.id}")
        }
        rv.setOnClickPendingIntent(
            R.id.widget_circle,
            PendingIntent.getBroadcast(
                context,
                todo.id.toInt(),
                complete,
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
            )
        )
        return rv
    }
}

/** Repository 的进程内单例（widget 侧专用，避免每个广播都新建）。 */
object TodoWidgetRepo {
    @Volatile
    private var instance: com.lirui.oneline.data.TodoRepository? = null

    fun get(context: Context): com.lirui.oneline.data.TodoRepository =
```

```
        instance ?: synchronized(this) {
            instance ?: com.lirui.oneline.data.TodoRepository(context.applicationContext)
                .also { instance = it }
        }
}

/**
 * 一行的到期文案：是否逾期。无日期返回空串 + false。
 *
 * 组件里**不带「周几」**（zuige 2026-10-23：只要日期），所以这里恒传 `showWeekday = false`；
 * App 列表仍用 `Format.dueLine` 的默认值（带星期）。
 * [showTime]=false 时只到日期（「10 月 23 日」）—— 对应设置里的「只显示日期」开关。
 * [endAtMillis] 非空 = 时间区间（#1）：文案变成「15 点~17 点」，逾期看结束点。
 */
internal fun dueLabel(
    dueAtMillis: Long?,
    hasExactTime: Boolean,
    showTime: Boolean = true,
    startAtMillis: Long? = null,
    endAtMillis: Long? = null
): Pair<String, Boolean> {
    if (dueAtMillis == null) return "" to false
    val today = AppTime.today()
    val now = AppTime.nowMillis()
    val overdue = Format.isOverdue(endAtMillis ?: dueAtMillis, today, now, hasExactTime)
    val label = when {
        !showTime -> if (startAtMillis != null) {
            Format.monthDayRange(startAtMillis, dueAtMillis)
        } else {
            Format.dueLine(dueAtMillis, showWeekday = false, showTime = false)
        }
        // 时间段（日期段）用紧凑范围（21–23 日），时间区间再接「15 点~17 点」。
        startAtMillis != null -> {
            val range = Format.monthDayRange(startAtMillis, dueAtMillis)
            val t = if (endAtMillis != null) {
                "${Format.time(dueAtMillis)}~${Format.time(endAtMillis)}"
            } else {
                Format.time(dueAtMillis)
            }
            "$range $t"
        }
        // 同一天的时间区间：「10 月 23 日 15 点~17 点」。
        endAtMillis != null -> {
            val d = Format.dueLine(dueAtMillis, showWeekday = false, showTime = false)
            "$d ${Format.time(dueAtMillis)}~${Format.time(endAtMillis)}"
        }
        else -> Format.dueLine(dueAtMillis, showWeekday = false, showTime = true)
    }
    return label to overdue
}
```

（以上为源程序后连续 30 页，每页 50 行）